NCP TECHNICAL REPORT

NCP System Design

Exact exchanges, bounded state, and evidence boundaries for independently owned simulation and control software

Sepehr Mahmoudian

27 September 2026

UNRELEASED • MODULAR SDK IMPLEMENTED • CANDIDATE 1.0.0-rc.1 RELEASE-BLOCKED • NO RELEASE AUTHORITY

Cite this report as: Sepehr Mahmoudian (2026). *NCP System Design.* NCP repository report. Include the exact repository commit or release that you used. The software citation metadata is in `CITATION.cff`.

Abstract

NCP (Neuro-Cybernetic Protocol) connects independently owned simulations, bodies, sensors, and analysis tools through exact, bounded data contracts. This report describes NCP in three scopes. The modular local SDK is implemented twice, in Rust and in Python. It retains each outcome until an exact acknowledgement. It moves payloads in bounded chunks, and it checks the digest of each chunk and of each complete payload. Selected installed campaigns observed it. A fixed-role local profile serves as a regression control. The broader `1.0.0-rc.1` candidate runs wire `1.0` today, and a proposed architecture closes its known gaps. That architecture separates four traffic planes, admits each frame through ordered gates, and bounds every retained byte.

The report derives each interface and resource equation from named assumptions. A Lean 4 file checks the discrete arithmetic. The equations are design accounting, not measurements. No part of NCP is released. Local tests, equations, and figures cannot establish live security, physical safety, controller stability, or scientific validity.

CLAIM BOUNDARY

Source review, equations, figures, PDF reproduction, tests, and semantic models are local evidence. They cannot establish live security, independent interoperability, physical safety, or controller stability. They also cannot establish performance qualification, signed provenance, or release readiness. A local result stays inside its named evidence class.

Contents

1 Scope and reading rules . . . 3
2 Notation . . . 5
3 Modular exchange . . . 6
4 Bounded payloads and logical capacity . . . 8
5 Applications and observed evidence . . . 10
6 Fixed-role reference profile . . . 12
7 The 1.0 candidate and its target . . . 13
8 Admission and planes . . . 13
9 Prepared runtime and hot-path latency . . . 15
10 Finite memory and queue isolation . . . 16
11 Session lifecycle . . . 18
12 Streams, positions, and fixed layouts . . . 20
13 Simulation steps . . . 21
14 Plant command admission . . . 23
15 Effect boundary and closed-loop limits . . . 24
16 Extensions and the complete state envelope . . . 26
17 Identity and ecosystem . . . 27
18 Current candidate runtime and gaps . . . 29
19 Ten-lens review . . . 30
20 Evidence classes and release gates . . . 31
A How this report is checked . . . 32
B Equation audit . . . 33
C Glossary . . . 34

1 Scope and reading rules

1.1 What NCP does

NCP is a contract layer between separate programs. It defines message identity, the correlation of one request with one outcome, acknowledgement, and the lifetime of byte buffers. Each program keeps its own state, physics, models, and decisions. NCP runs no central process.

Figure 1 shows the implemented modular composition. A host program selects the applications, installs their exact contracts, and connects each application through NCP. The applications in the figure are the current ecosystem projects. Engram hosts a network in NEST, a simulator for spiking neural networks. CREBAIN simulates bodies and sensors. Prisoma captures exchanges and orders experiments. Galadriel checks sensor consistency.

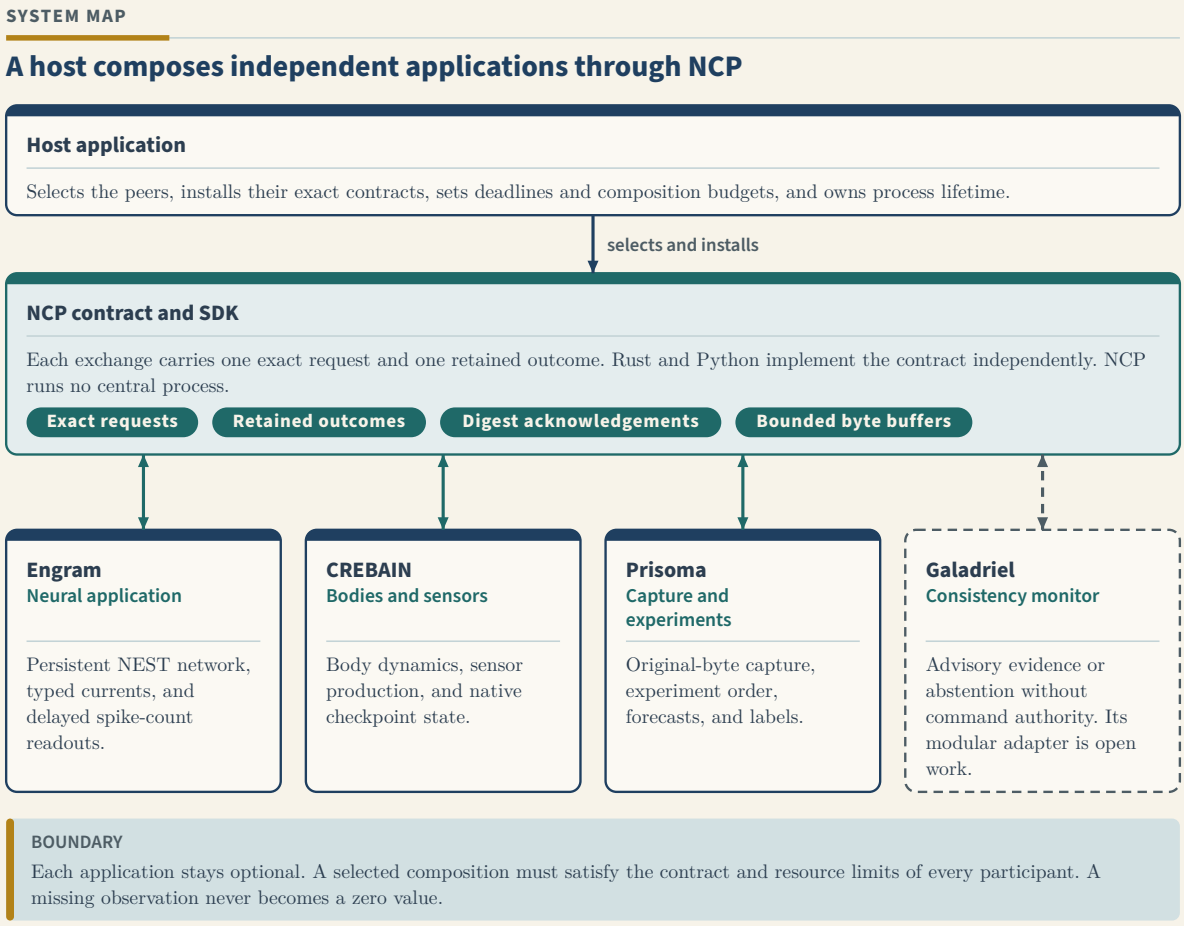


Figure 1: The modular system. The host selects independent applications. Each exchange carries one exact request and one retained outcome.

1.2 Three scopes

NCP has three scopes with different identities and evidence. Table 1 lists them. Do not transfer a result from one scope to another.

Table 1: The three NCP scopes.

Scope	Identity	State and evidence
Modular local SDK	Descriptor `ncp-core/src/modular_profile.v1.json`	Implemented in Rust and Python. Source controls, installed-wheel checks, and selected native campaigns exist.
Fixed-role reference profile	`ncp.local-lockstep.v1`, intended tag `local-v1.0.0`	Implemented regression control with four fixed owners. Its publication gates are open.
Broader protocol candidate	Package `1.0.0-rc.1`, wire `1.0`	The wire runs today. A proposed architecture closes its known gaps. Ten external release gates are **NOT RUN**.

1.3 Status labels

Each section starts with one status label. The labels have these meanings:

- **Implemented:** source exists and its local controls pass. The label says nothing about installed or live behavior.
- **Observed:** a dated installed campaign retained the result for its exact artifacts, configuration, and workload.
- **Proposed:** design only. It needs independent review, identifier allocation, implementation, and conformance evidence.
- **NOT RUN:** a required gate has no exact evidence.

Figure 2 orders the evidence classes behind these labels. Implemented work has evidence of class 1 or 2. Observed work adds class 3. The class 4 gates are **NOT RUN**, and no class 5 release exists. A lower class never promotes a claim to a higher class.

1.4 Identities

The candidate package version is `1.0.0-rc.1`. Its wire version is `1.0`. The compact protobuf hash `163acc57d8a62b66` is a 64-bit FNV-1a digest of the canonical protobuf structure. It is advisory only. The complete contract digest is this SHA-256 value over 30 normative sources:

`9cae331742d01e9b164e029aa06c644e6b1886176d0816a6ef883af138355c90`

The file `contract/manifest.v1.json` lists the sources. The latest immutable release is `v0.8.0`. It uses wire `0.8`, which is a different protocol.

This edition describes the repository at commit `e83df54` of 27 September 2026. Paths in this report are relative to the repository root. Appendix A lists the checks that rebuild and verify this report.

1.5 Changes since the previous edition

The previous edition described only the proposed architecture. Since then, these changes landed:

- Independent Rust and Python SDKs for the fixed-role profile (5 September 2026).
- The modular local SDK and its installed-package gates (7 September 2026).
- The TypeScript WebSocket binding now bounds queued payload bytes at 8,388,608 in addition to 128 pending requests (21 September 2026). Bytes are released when `WebSocket.send` returns, so browser and kernel buffers are outside this bound.

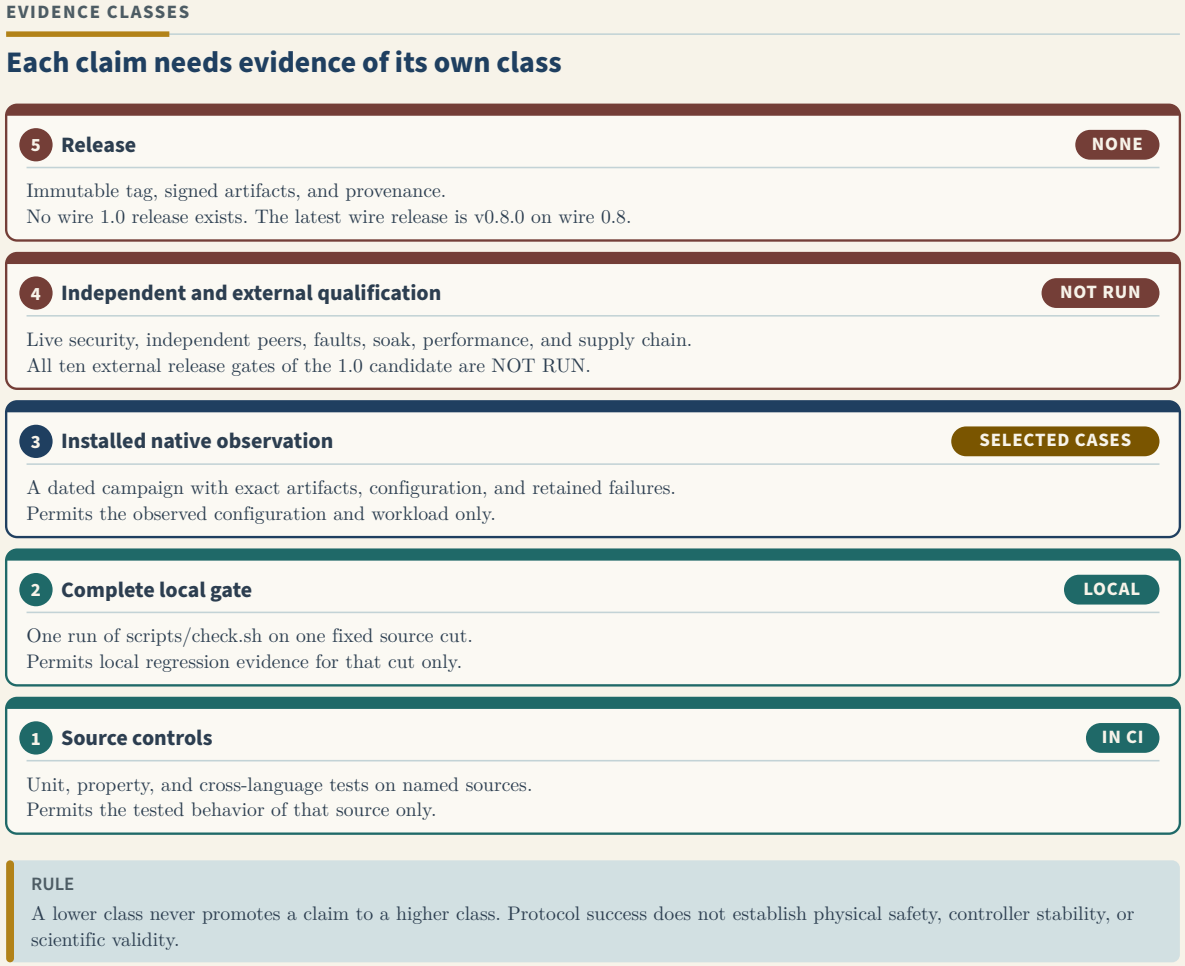


Figure 2: Evidence classes and the claims that each class permits.

- The TypeScript client keeps one mutation in flight for each generation and retires a generation after an uncertain outcome (21 September 2026).
- Selected installed campaigns of the modular applications, from 23 to 27 September 2026 (Table 5).

2 Notation

A hat marks a declared upper bound: $\hat{x}$ is the largest value that a profile admits for x. Table 2 lists the symbols that more than one section uses. Each section defines its other symbols where they first occur.

Let W be the largest value of the declared integer type. Checked addition is the partial function

$$\operatorname{cadd}(x, y) = \begin{cases} x + y, & x + y \le W, \\ \text{undefined}, & x + y > W. \end{cases} \tag{1}$$

An undefined result never wraps or saturates. The owner rejects the operation or seals the affected counter before it changes state. Clock ticks from different incarnations never meet in one expression.

Table 2: Shared notation.

Symbol	Meaning
$\mathbb{N}$, $\mathbb{N}_{>0}$	Nonnegative and positive integers
$\lvert m\rvert$	Length in bytes of the byte string m
$\#X$	Number of elements of the finite set X
$\lceil q\rceil$, $\lfloor q\rfloor$	Smallest integer $\geq q$, largest integer $\leq q$
$H(m)$	SHA-256 digest of m. A digest locates a record. Exact byte comparison still decides equality.
$\mathrm{cadd}(x, y)$	Checked addition, defined in equation (1)
t with a subscript	A tick of one receiver-owned monotonic clock incarnation
$\hat{x}$	Declared upper bound of x

3 Modular exchange

Status: implemented.

Rust and Python implement this contract independently. The owners are `ncp-core/src/modular_owner.rs` and `local/python/ncp_local/modular_owner.py`. Their shared descriptor is `ncp-core/src/modular_profile.v1.json`.

3.1 Installed application boundary

The host selects an application before it opens a channel. The application supplies closed types for preparation, commands, imports, metadata, finish, and results. It also supplies validators and its exact descriptor. Each message binds the digests of the core descriptor and the application descriptor.

A request cannot install a schema, load a module, select an executable, or open a path that a peer selects. The execution interface accepts no method string with an arbitrary JSON value. An unknown application identity fails admission.

3.2 Request identity

Each request names a binding, a sequence number, a closed operation, and the complete typed digest. The binding names the profile, application, run, endpoint, and generation. The sequence number s satisfies

$$s \in \{1, 2, \ldots, \hat{s}\}, \qquad \hat{s} = 2^{53} - 1 = 9{,}007{,}199{,}254{,}740{,}991. \tag{2}$$

The bound $\hat{s}$ is the largest integer n for which binary64 represents both n and $n + 1$ exactly. Every integer from 0 to 2^{53} is exact in binary64, and every larger integer parses to at least 2^{53}. A reader that parses numbers as binary64 therefore keeps each valid s exact and still detects an out-of-range value. The codec rejects `-0`, fractions, booleans, and larger integers.

Let r_s be the retained outcome bytes of request s. The first request has no predecessor. Every later request must name the digest of the last admitted outcome:

$$\mathrm{pred}(s + 1) = H(r_s). \tag{3}$$

Queries and acknowledgements do not change the predecessor.

3.3 Owner steps

Figure 3 shows the order of owner work. The owner checks for an exact duplicate before it reads mutable application state. The first execution can change the state that admitted it, so a later state check could reject a valid retry.

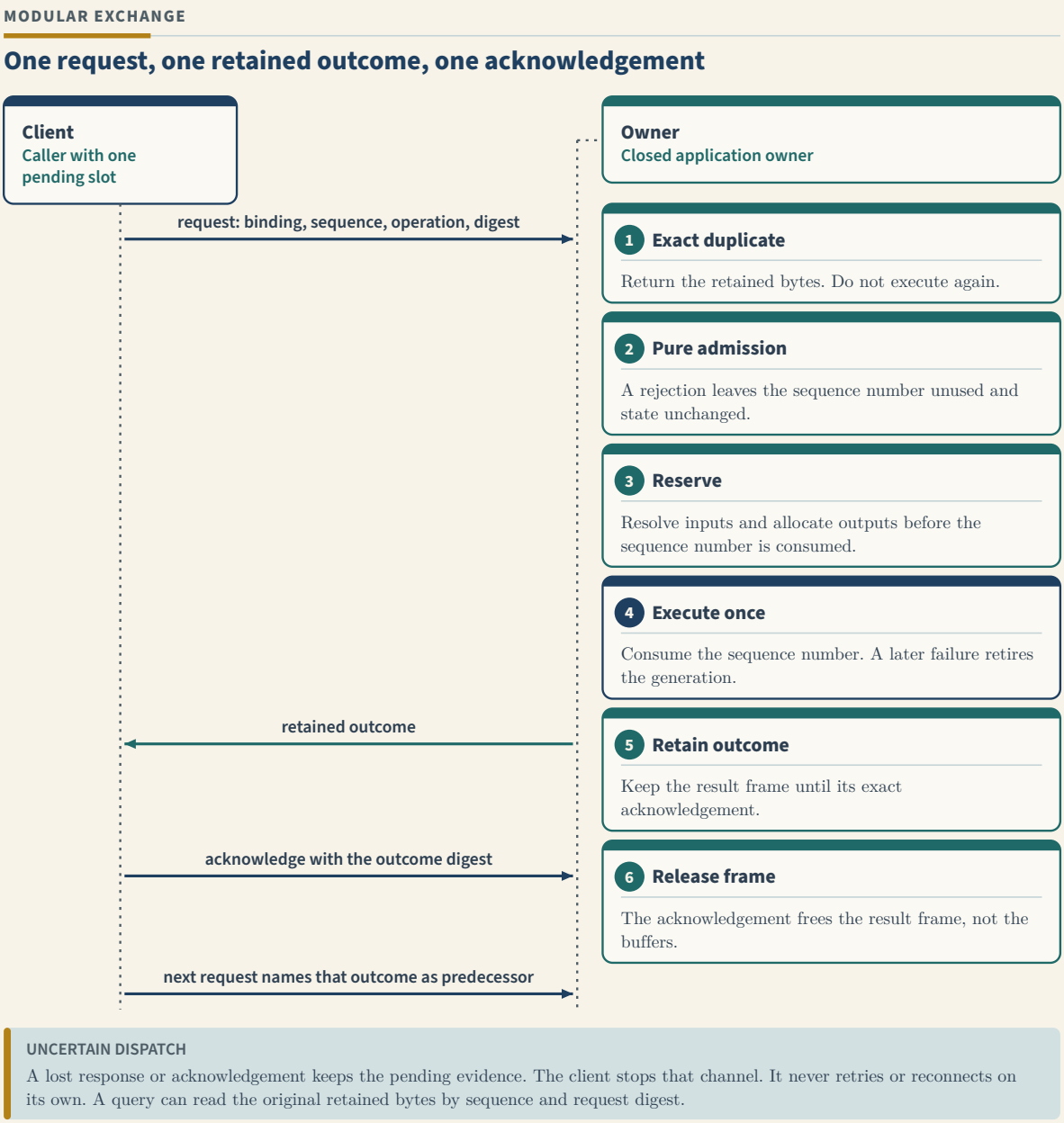


Figure 3: The modular exchange. An exact duplicate returns the retained bytes. Admission changes nothing. Reservation precedes the use of the sequence number. The acknowledgement frees only the result frame.

Admission is pure: it reads state and returns the demanded inputs and outputs. It performs no I/O, external reservation, or backend mutation. The core then resolves each input and allocates each output. Only after that step does it consume s and assign buffer identifiers.

A backend error, panic, invalid result, or incomplete output after entry consumes s. The owner retains a bounded indeterminate result when possible and retires the generation. It never converts that outcome into success or into a clean rejection.

Before commit, the owner decodes the staged response with the installed output codec. The decoded value must reproduce the complete typed digest. This check rejects a non-finite value that a codec turns into `null`, and it rejects signed-zero normalization.

3.4 Worked example

Preparation at $s = 1$ produces outcome r_1. After the client acknowledges r_1, it sends $s = 2$ with predecessor $H(r_1)$. Suppose admission rejects the input of request 2. The rejection leaves $s = 2$ unused and keeps the predecessor $H(r_1)$. The client can send corrected input as $s = 2$ without an acknowledgement of the rejection. A conflict, an unavailable result, or a mismatched response cannot use this path.

3.5 Uncertain dispatch

End of stream, partial framing, a timeout, or a failed read or write retires the channel. A lost acknowledgement keeps the pending evidence. The client stops the channel and never retries or reconnects on its own. A query can read the original retained bytes by sequence number and request digest.

4 Bounded payloads and logical capacity

Status: implemented.

The payload primitives are `ncp-core/src/modular_buffer.rs` and `local/python/ncp_local/modular_buffer.py`.

4.1 Chunked transfer

A payload is one sealed byte string of length ℓ with $1 \le \ell \le \hat{\ell}$ and $\hat{\ell} = 8{,}388{,}608$ bytes. A chunk carries at most $\hat{\ell}_c = 32{,}768$ decoded bytes. The chunk count is

$$m = \left\lceil \frac{\ell}{\hat{\ell}_c} \right\rceil . \tag{4}$$

Chunk j, for $0 \le j < m$, starts at byte $j\hat{\ell}_c$ and has length

$$\ell_j = \min(\hat{\ell}_c,\ \ell - j\hat{\ell}_c). \tag{5}$$

Derivation 4.1 (Chunks tile the payload)**.** Equation (4) gives $(m-1)\hat{\ell}_c < \ell \le m\hat{\ell}_c$. For $j \le m-1$, the start satisfies $j\hat{\ell}_c \le (m-1)\hat{\ell}_c < \ell$, so $\ell_j \ge 1$. Every chunk before the last has length $\hat{\ell}_c$. The last chunk has length $\ell - (m-1)\hat{\ell}_c$. The lengths therefore sum to ℓ with no gap or overlap. The payload bound gives

$$m \le \left\lceil \frac{\hat{\ell}}{\hat{\ell}_c} \right\rceil = 256. \tag{6}$$

Figure 4 shows the transfer. The receiver reserves all ℓ bytes before it accepts the first chunk. It admits only the next contiguous chunk. Each chunk must match its index, offset, decoded length, canonical base64 spelling, and SHA-256 digest. The seal then checks the SHA-256 digest of the complete payload. Valid chunk digests cannot replace that check. A failed admission keeps counters, reservations, and stored bytes unchanged.

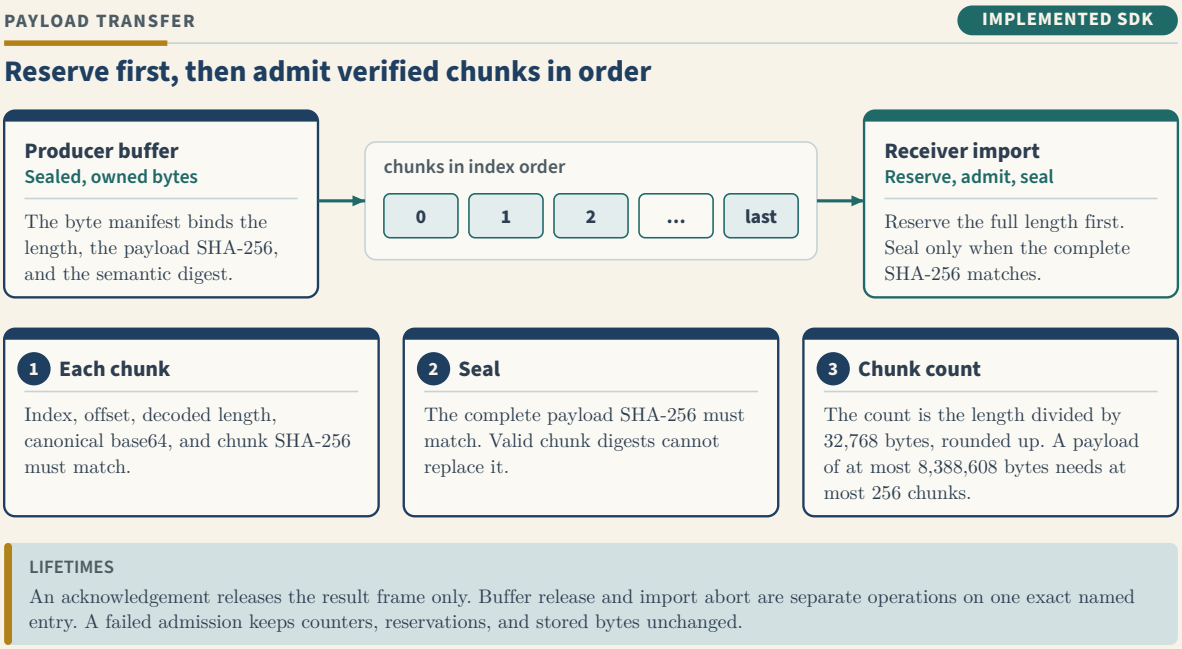


Figure 4: Bounded payload transfer. The receiver reserves the full length first. The seal checks the complete digest.

An acknowledgement releases the retained result frame only. Buffer release and import abort are separate operations on one exact named entry.

4.2 Working-state bounds

Table 3 lists the bounds in the core descriptor `ncp-core/src/modular_profile.v1.json`. They are logical reservations. They do not bound process memory, allocator overhead, or graphics memory.

Table 3: Modular SDK bounds from the core descriptor.

Item	Bound
JSON frame $\hat{\ell}_F$, after a 4-byte length prefix	65,536 bytes
Decoded chunk $\hat{\ell}_c$	32,768 bytes
One payload $\hat{\ell}$	8,388,608 bytes
Payload reservations for one endpoint	67,108,864 bytes
Live entries for one endpoint (sealed and incomplete)	24, of which at most 12 incomplete
Endpoints and payload reservations in one composition	16 and 268,435,456 bytes
Serialized byte manifest	4,096 bytes
Installed semantic-digest roster	64 SHA-256 digests
Sequence numbers and buffer identifiers	at most $2^{53} - 1$

The owner reserves one fixed logical working extent for each endpoint. Its metadata categories are core metadata (131,072 bytes), import metadata (32,768 bytes), and selected-input metadata (16,384 bytes). Its scratch categories are digest and sort scratch (131,072 bytes), one decoded chunk, and 14 frame-sized extents. The 14 extents are 6 complete frames, 3 parsed representations, 1 decoded Unicode source, 2 scanner scratch extents, and 2 scalar scratch extents. The owner total is

$$R_{\text{own}} = 131{,}072 + 32{,}768 + 16{,}384 + 131{,}072 + \hat{\ell}_c + 14\,\hat{\ell}_F = 1{,}261{,}568 \text{ bytes}. \tag{7}$$

The client has no pool metadata but keeps 4,096 bytes of fixed state:

$$R_{\text{cli}} = 131{,}072 + \hat{\ell}_c + 4{,}096 + 14\,\hat{\ell}_F = 1{,}085{,}440 \text{ bytes}. \tag{8}$$

Sixteen owners reserve $16R_{\text{own}} = 20{,}185{,}088$ bytes, and sixteen clients reserve $16R_{\text{cli}} = 17{,}367{,}040$ bytes. Payload reservations are separate. The canonical base64 text of one full chunk has $4\lceil \widehat{\ell}_c/3 \rceil = 43{,}692$ bytes. Its two transient copies need 87,384 bytes, which fit in the two scalar extents ($2\widehat{\ell}_F = 131{,}072$ bytes).

5 Applications and observed evidence

Status: observed for the selected campaigns only.

5.1 Ownership

Table 4: What each component owns and does not supply.

Component	Owns	Does not supply
NCP	Bounded frames, exact requests, retained outcomes, acknowledgements, byte buffers, and two independent codecs	Physics, sensor meaning, process launch, experiment design, or model quality
CREBAIN	Body dynamics, selected sensors, typed observations, and the installed producer lifecycle	Neural execution or a learned world model
Engram	A persistent NEST network, typed currents, and delayed spike-count readouts	Body physics, capture, or monitoring
Prisoma	Original-byte capture, experiment order, forecasts, and restored labels	Simulator ownership or model-quality qualification
Host	Selected peers, bindings, deadlines, and composition budgets	Authority over an unknown application contract

Each application stays optional. CREBAIN runs without Engram, Prisoma, or a monitor. A selected composition must satisfy the contract and resource limits of each participant. A missing observation never becomes a zero value.

5.2 Sensor clock arithmetic

The CREBAIN body advances at $f_b = 120$ ticks per second. Its acoustic model produces $f_a = 16{,}000$ samples per second. For body tick $k \geq 1$, the pressure samples have the index interval

$$I_k = \left[\left\lfloor \frac{(k-1)f_a}{f_b} \right\rfloor, \left\lfloor \frac{kf_a}{f_b} \right\rfloor \right). \tag{9}$$

The ratio $f_a/f_b = 400/3$ is not an integer. The first three ticks contain 133, 133, and 134 samples. For every $u \in \mathbb{N}$,

$$\sum_{k=3u+1}^{3u+3} \#I_k = \left\lfloor \frac{(3u+3)\cdot 400}{3} \right\rfloor - \left\lfloor \frac{3u \cdot 400}{3} \right\rfloor = 400. \tag{10}$$

Each group of three ticks therefore holds exactly 400 samples and spans 25 ms of simulated time. The endpoints are sample indices, not timestamps. The CREBAIN contract `crebain.sensor-pressure.v1` stores each sample as one 8-byte little-endian binary64 value in pascals.

Two retained campaigns in Table 5 match this partition. The sensor transfer counted 3,200 pressure samples in 24 body ticks, which is eight groups of 400. The city maximum batch recorded the

intervals $[0, 133)$, $[133, 266)$, and $[266, 400)$ for ticks 1 to 3. Its third tick is 32 bytes larger, because each of four microphones carries one more sample.

5.3 Neural coupling arithmetic

The example policy advances NEST once per three body ticks, with step $\Delta_n = 25{,}000\,\mu$s and recording delay $\delta = 1{,}000\,\mu$s. For an executed end time t_e, the complete spike window is $(\tau_0, \tau_1]$ with

$$\tau_0 = \max(0,\ t_e - \Delta_n - \delta), \qquad \tau_1 = t_e - \delta. \tag{11}$$

All times are integer microseconds. For $t_e \geq \Delta_n$ and $\delta < \Delta_n$, the window is nonempty: $\tau_1 - \tau_0 = \min(t_e - \delta,\ \Delta_n) > 0$. For n_{ne} neurons and n_{sp} spikes in the window, the mean rate for each neuron is

$$\text{rate} = \frac{n_{\text{sp}} \cdot 10^6}{n_{\text{ne}}(\tau_1 - \tau_0)}\ \text{Hz}. \tag{12}$$

A complete zero count is observed silence. A pending window supplies no count and no rate. This discrete policy tests interface causality. It makes no learned-control claim.

5.4 Observed campaigns

Table 5 lists selected dated campaigns from `local/modular/STATUS.md`. Each row holds only for its exact artifacts, configuration, and workload. Observations from different identities do not combine into one release campaign.

5.5 Remaining modular work

Each of these items needs its own evidence in its owning application:

- installed qualification of each advertised combination,
- a measured resource envelope,
- many-entity operating evidence, including the failed 7,200-tick horizon,
- a qualified modular monitor,
- model or policy benefit beyond the null forecast result, and
- a scoped product publication.

Table 5: Selected installed campaigns (September 2026).

Campaign	Retained observation	Limit
Sensor transfer (Prisoma)	Installed body-only and canonical runs matched all 44 payloads and 4,326,400 bytes. Record: `M1_NATIVE_2026-09-23.md`.	Fault cases keep incomplete captures and their original cleanup results.
Engram host (Engram)	Two arms completed 24 body ticks and eight NEST steps. A fresh direct NEST run matched all 64 count and rate values. Record: `NATIVE_HOST_EVIDENCE.md`.	The reference shares the NEST numerical engine. Paper reproduction is not qualified.
Restored-label study (Prisoma)	Eight qualification cases and 112 study episodes completed. Record: `E1_NATIVE_2026-09-23.md`.	The forecast result is null or inconclusive. The constant comparison missed the frozen useful margin.
City source delivery (CREBAIN)	Native delivery and fault cases retained 153 payloads with 30,121,184 bytes. Record: `NATIVE_CITY_2026-09-23.md`.	Delivery does not show contact absence or complete resource use.
City maximum batch (CREBAIN)	Three ticks retained 83,571,200 bytes. Tick three reached the 27,857,088-byte batch ceiling. Record: `NATIVE_CITY_BOUNDARY_2026-09-26.md`.	One maximum batch does not cover every horizon or all memory.
City horizon (CREBAIN)	A second maximum batch completed. The 7,200-tick CPU case failed at its 600-second deadline after 4,070 acknowledged ticks. Record: `native-city-horizon-2026-09-27.md`.	The complete city profile and long horizons are not qualified.
City CPU and storage (CREBAIN)	Six one-drone cases verified 216 payloads with 230,400 bytes. All 144 route deadlines and all 144 export deadlines were missed. Record: `native-city-cpu-storage-2026-09-27.md`.	One shared host and 24 ticks for each case isolate neither overhead nor tail latency.
Timing (Prisoma)	All 192 executions completed. All 4,608 route deadlines and all 4,608 export deadlines were missed. Record: `M1_PERFORMANCE_2026-09-23.md`.	No real-time claim follows.

Prisoma records are in `integrations/agent-bridge/evidence/`. CREBAIN records are in `integrations/ncp-force-city-sources/evidence/`. The Engram record is in `packages/ncp-nest/docs/` of the private Paper2Brain repository.

6 Fixed-role reference profile

Status: implemented as a regression control, with open publication gates.

The profile `ncp.local-lockstep.v1` fixes four owners. Engram coordinates the experiment and owns one NEST network. CREBAIN owns the simulated bodies. Prisoma reserves capture capacity and records exact step pairs. Galadriel runs its statistical detector in record-only mode. Each owner uses a private process pipe. There is no remote listener.

A plan has one to three entities and at most 1,024 coupled steps. A step lasts 1,000 to 1,000,000 μs in multiples of 1,000 μs. A frame is a 4-byte big-endian length followed by at most 65,536 JSON bytes. Positions use meters, velocities meters per second, and actions meters per second squared, in east, north, and up order. The action `zero_acceleration` commands zero acceleration, so it can keep the current velocity. It is not a stop command.

Capture reservation precedes neural and body mutation. A lost response does not show whether execution occurred. The coordinator keeps known results, records unresolved dispatches, and retires uncertain generations. Same-generation crash resume is out of scope. Haldir command gating,

remote endpoints, physical actuation, and real-time guarantees are also out of scope. The registry `local/release.v1.json` lists the open bootstrap and operational gates.

7 The 1.0 candidate and its target

Status: current wire implemented, target architecture proposed.

The candidate implements wire `1.0` in Rust (`ncp-core`) and in an independent TypeScript client (`ncp-ts`). It also has a Zenoh transport (`ncp-zenoh`), a gateway, and Python and C/C++ bindings over Rust. Its mandatory corpus has 282 vectors. Zenoh is a publish-subscribe and query transport.

Sections 8 to 17 describe the proposed target. It closes the gaps in Section 18. Eleven proposed decision records in `docs/adr/` define it. Before implementation starts, three steps must pass in order:

1. Independent reviewers accept the eleven decision records.
2. The project owner authorizes a deliberate rebaseline of the candidate.
3. A registry step allocates exact identifiers, error codes, and numeric limits.

The task ledger `docs/implementation/NCP_1_0_TASK_LEDGER.md` tracks these steps. Table 12 in Appendix C maps its task codes to these names. Until the three steps pass, every symbolic bound below has no selected value.

8 Admission and planes

Status: proposed.

8.1 Four core planes

NCP has exactly four core planes. Table 6 lists them. Extension traffic uses a separate namespace and resource class. It is not a fifth plane and inherits no plane authority or capacity.

Table 6: Core planes and overload policy.

Plane	Publisher	Consumer behavior	Overload
Control	Authenticated role authority	Bounded request, idempotent mutation, retained receipt	Reject
Perception	Body	The newest valid sample wins	Replace the latest
Action	Lease holder or enrolled emergency source	Direct body admission	One body priority slot
Observation	Body or declared producer	Read-only delivery	Drop the oldest and count

8.2 Ordered gates

Figure 6 shows the gates with the numbers 1 to 6. Let f be one received frame and $\gamma_j(f)$ the Boolean result of gate j:

$$(\gamma_1, \dots, \gamma_6) = (\gamma_{\text{raw}}, \gamma_{\text{principal}}, \gamma_{\text{wire}}, \gamma_{\text{session}}, \gamma_{\text{typed}}, \gamma_{\text{effect}}). \tag{13}$$

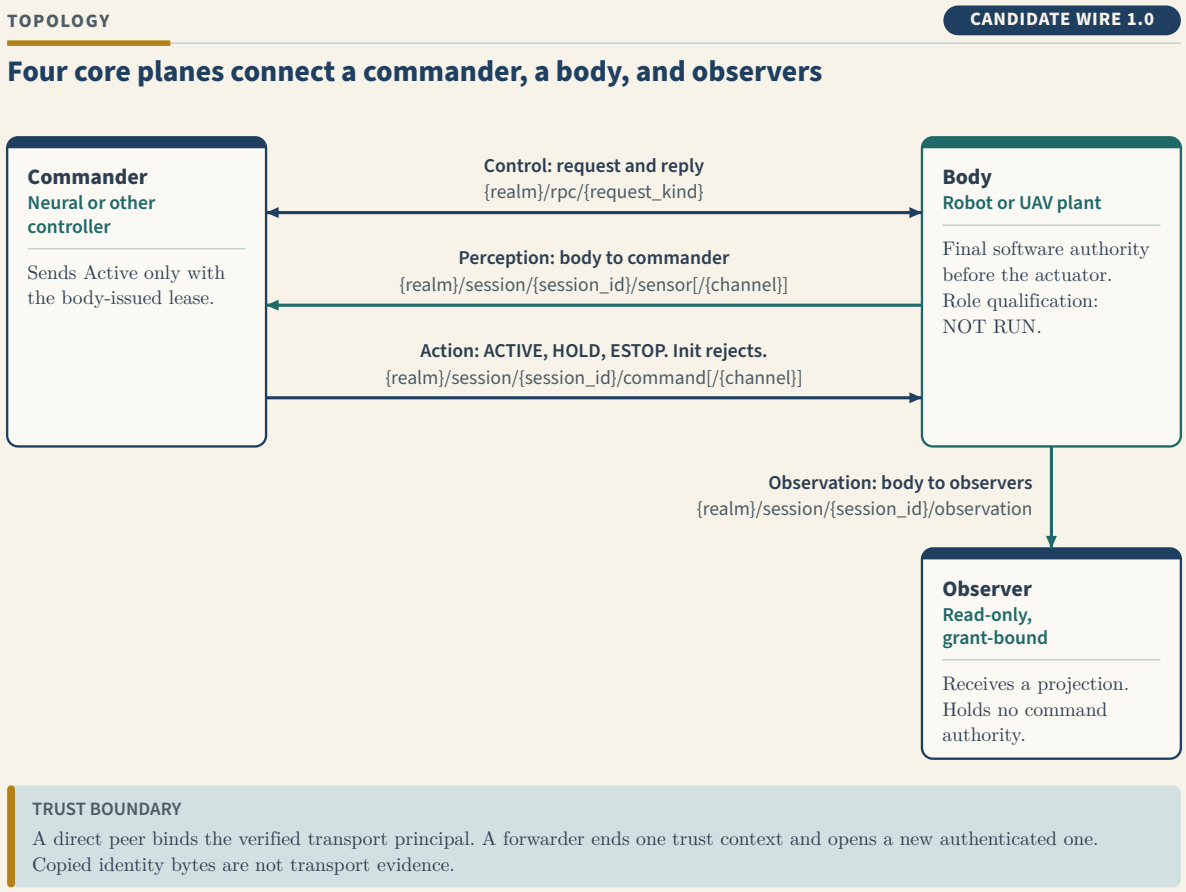


Figure 5: Topology. The routes are the current wire `1.0` key grammar. The body is the final software authority before the actuator.

Gates 1 to 5 are shared. For a non-action frame, $\gamma_{\text{effect}}(f)$ is true by definition. For an action frame, it is body-owned admission at the effect boundary. A frame is admitted only when every gate is true:

$$\Gamma(f) = \bigwedge_{j=1}^{6} \gamma_j(f). \tag{14}$$

The receiver does not evaluate equation (14) as an unordered set. It evaluates the prefixes

$$\Gamma_0(f) = \text{true}, \qquad \Gamma_j(f) = \Gamma_{j-1}(f) \wedge \gamma_j(f), \quad 1 \le j \le 6, \tag{15}$$

and runs gate j only when $\Gamma_{j-1}(f)$ is true.

Derivation 8.1 (First failure decides)**.** Suppose the ordered evaluation meets a false gate. Let j^* be the first index with $\gamma_{j^*}(f) = \text{false}$. Then $\Gamma_{j^*-1}(f) = \text{true}$ and $\Gamma_{j^*}(f) = \text{false}$. No gate after j^* runs. The rejection class of gate j^* is the result. The receiver emits no diagnostic that depends on an unbounded or unauthenticated later field.

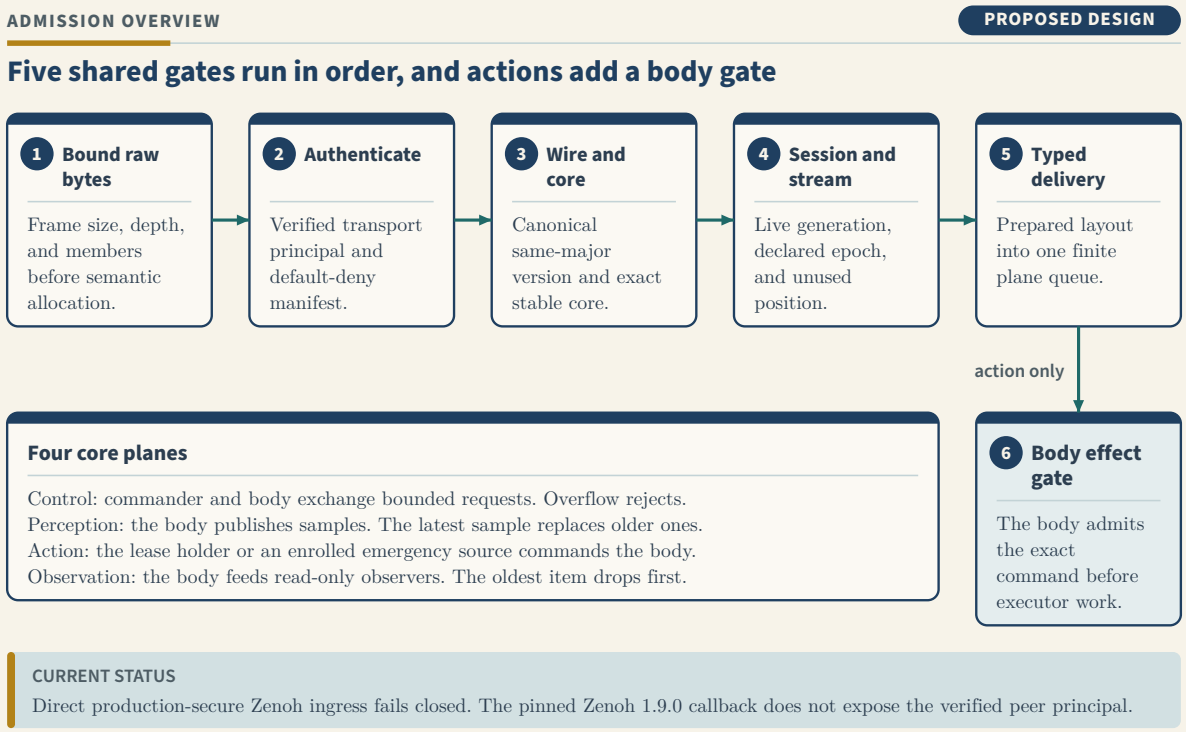


Figure 6: Five shared gates run in order. Action frames add the body effect gate.

9 Prepared runtime and hot-path latency

Status: proposed.

Preparation validates deployment input once. It compiles immutable handles and reserves shared state before any publisher, subscription, callback, or actuator resource exists. Figure 7 shows the six stages of each later frame.

9.1 Stage times

Let the ordered stages be

$$\mathcal{J} = (\text{bound}, \text{verify}, \text{decode}, \text{locate}, \text{admit}, \text{handoff}). \tag{16}$$

For stage $i \in \mathcal{J}$, let t_i^{in} and t_i^{out} be ticks of one receiver clock incarnation. The stage time is

$$T_i = t_i^{\text{out}} - t_i^{\text{in}} \geq 0. \tag{17}$$

The accounting boundaries are contiguous: t_i^{out} equals $t_{i'}^{\text{in}}$ for the next stage i'. This equality is an accounting rule. It does not require a thread or process boundary. Every inner tick then occurs once with each sign, and the sum telescopes:

$$T_{\text{hot}} = t_{\text{handoff}}^{\text{out}} - t_{\text{bound}}^{\text{in}} = \sum_{i \in \mathcal{J}} T_i. \tag{18}$$

If a profile selects a finite ceiling $\hat{T}_i$ with $0 \leq T_i \leq \hat{T}_i$, then

$$T_{\text{hot}} \leq \sum_{i \in \mathcal{J}} \hat{T}_i = \hat{T}_{\text{hot}}. \tag{19}$$

Transport delay comes before $t_{\text{bound}}^{\text{in}}$. Queue, application, and device time come after $t_{\text{handoff}}^{\text{out}}$. None of them is part of T_{hot}. A later measurement must show that the implementation meets each $\hat{T}_i$ on the qualified platform.

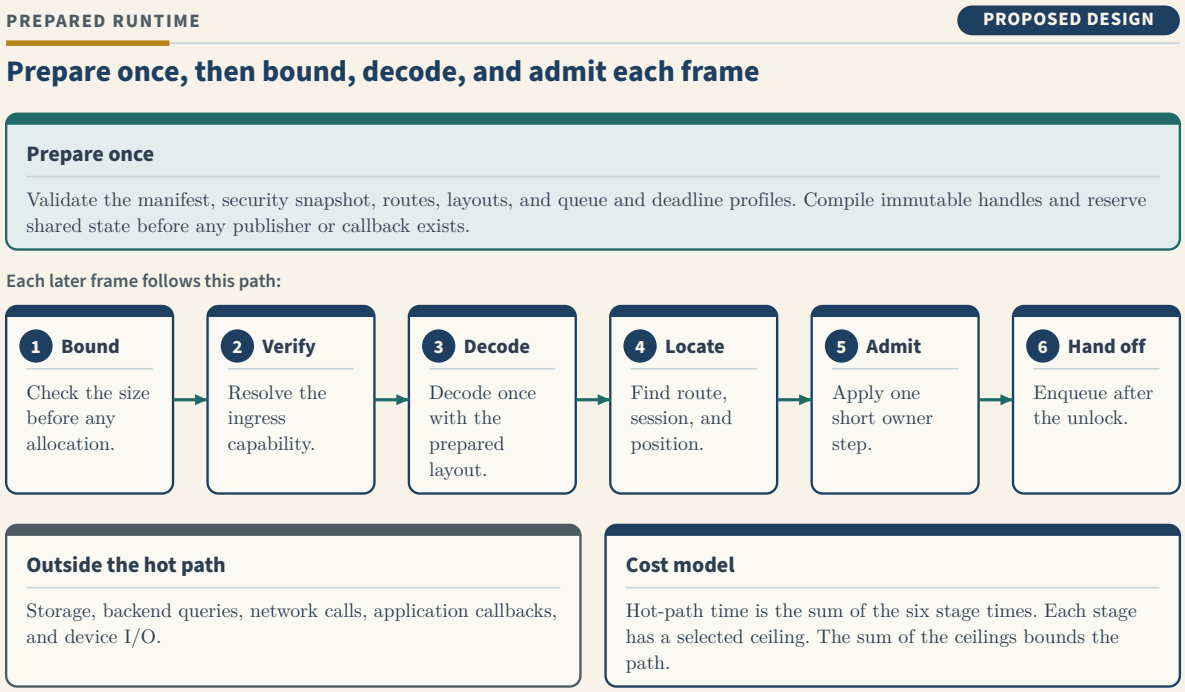


Figure 7: Prepared runtime. One short owner step admits each frame. External work runs outside the lock.

9.2 Allocations and copies

Let $n_{\mathrm{alloc},i}$ be the number of heap allocations and $n_{\mathrm{copy},i}$ the number of full-payload copies in stage i after preparation. The totals are

$$n_{\mathrm{alloc}} = \sum_{i \in \mathcal{J}} n_{\mathrm{alloc},i}, \qquad n_{\mathrm{copy}} = \sum_{i \in \mathcal{J}} n_{\mathrm{copy},i}. \tag{20}$$

For a fixed-layout frame in a contiguous receive buffer, the target is $n_{\mathrm{alloc}} = 0$ and $n_{\mathrm{copy}} = 0$. A segmented receive can need one bounded flattening copy. If preparation reserved that buffer, $n_{\mathrm{alloc}} = 0$ and $n_{\mathrm{copy}} \le 1$. These are targets, not current measurements.

10 Finite memory and queue isolation

Status: proposed.

10.1 Lifecycle stores

Let $\mathcal{S} = \{\mathrm{ses}, \mathrm{ctx}, \mathrm{res}, \mathrm{wait}\}$ name the session slots, prepared contexts, retained results, and optional waiters. Store $\kappa \in \mathcal{S}$ holds at most $\hat{n}_\kappa$ units. One unit is charged at most $\hat{b}_\kappa$ bytes, which cover content, backing storage, index entries, and metadata. The fixed owner overhead is $\hat{b}_0$ bytes. The bytes M_κ that store κ retains then satisfy

$$M_\kappa \le \hat{n}_\kappa \hat{b}_\kappa, \tag{21}$$

because units times bytes per unit gives bytes. The stores are disjoint accounting domains, so

$$M_{\mathrm{life}} \le \hat{b}_0 + \sum_{\kappa \in \mathcal{S}} \hat{n}_\kappa \hat{b}_\kappa. \tag{22}$$

With waiters off, $\hat{n}_{\text{wait}} = 0$, but $\hat{b}_0$ stays. An empty preallocated table does not cost zero bytes.

Fix one order $(\kappa_1, \dots, \kappa_4)$ of $\mathcal{S}$. The receiver computes each product and then the partial sums

$$\sigma_u = \hat{b}_0 + \sum_{v=1}^{u} \hat{n}_{\kappa_v} \hat{b}_{\kappa_v}, \qquad 1 \le u \le 4, \tag{23}$$

with checked arithmetic. An undefined product or sum rejects the profile before any allocation.

Portable bytes do not bound the heap of every language runtime. Each store therefore also has a finite, monotone local charge $J_\kappa(\hat{n}_\kappa)$ in one runtime unit, with fixed charge J_0:

$$J_{\text{life}} \le J_0 + \sum_{\kappa \in \mathcal{S}} J_\kappa(\hat{n}_\kappa). \tag{24}$$

The receiver admits a profile only when both bounds hold.

10.2 Plane queues

Let $\mathcal{Q} = \{\text{ctl}, \text{per}, \text{act}, \text{obs}, \text{ext}\}$ name the control, perception, action, observation, and extension queues. Queue $q \in \mathcal{Q}$ holds n_q items with $0 \le n_q \le \hat{n}_q$ and $\hat{n}_q \ge 1$. Its items $b_{q,1}, \dots, b_{q,n_q}$ retain

$$S_q = \sum_{k=1}^{n_q} |b_{q,k}| \le \hat{S}_q \tag{25}$$

bytes. Each item also carries at most μ_q metadata bytes. The visible-loss count satisfies $0 \le g_q \le \hat{g}_q$, where $\hat{g}_q \le W$ is the largest representable count.

Figure 8 shows admission of a new item b. The plane policy either rejects b or selects one allowed victim set $X_q(b) \subseteq \{1, \dots, n_q\}$. Control and extension select no victim. Perception can replace the latest replaceable item. Observation selects the oldest items. Action selects only work that its severity order permits it to displace. Define

$$e_q = \#X_q(b), \qquad R_q = \sum_{k \in X_q(b)} |b_{q,k}|. \tag{26}$$

The subset relation gives $0 \le e_q \le n_q$ and $0 \le R_q \le S_q$. Admission requires

$$n_q - e_q + 1 \le \hat{n}_q, \qquad S_q - R_q + |b| \le \hat{S}_q, \qquad g_q + e_q \le \hat{g}_q. \tag{27}$$

If no allowed victim set satisfies the first two conditions, the owner rejects b and leaves the queue unchanged. The owner computes the third sum as $\text{cadd}(g_q, e_q)$. If the loss count cannot advance, the owner seals the affected stream and leaves the queue unchanged. Otherwise one atomic owner step records the loss, removes the victims, and installs b. The new state is

$$n'_q = n_q - e_q + 1 \le \hat{n}_q, \qquad S'_q = S_q - R_q + |b| \le \hat{S}_q, \qquad g'_q = g_q + e_q \le \hat{g}_q. \tag{28}$$

A count limit alone is not enough, because item sizes vary. With fixed queue overhead $\hat{b}_{Q,0}$, the portable queue envelope is

$$M_{\text{queues}} \le \hat{b}_{Q,0} + \sum_{q \in \mathcal{Q}} (\hat{S}_q + \hat{n}_q \mu_q). \tag{29}$$

Stores and queues are disjoint domains. Their prepared total is

$$M_{\text{prep}} \le \hat{b}_0 + \sum_{\kappa \in \mathcal{S}} \hat{n}_\kappa \hat{b}_\kappa + \hat{b}_{Q,0} + \sum_{q \in \mathcal{Q}} (\hat{S}_q + \hat{n}_q \mu_q). \tag{30}$$

Each queue also has a finite local charge $J_q(\hat{n}_q, \hat{S}_q)$, monotone in both arguments, with fixed charge $J_{Q,0}$:

$$J_{\text{prep}} \le J_0 + \sum_{\kappa \in \mathcal{S}} J_\kappa(\hat{n}_\kappa) + J_{Q,0} + \sum_{q \in \mathcal{Q}} J_q(\hat{n}_q, \hat{S}_q). \tag{31}$$

No queue borrows the count, bytes, or local charge of another queue. Observation pressure therefore cannot consume reserved action capacity.

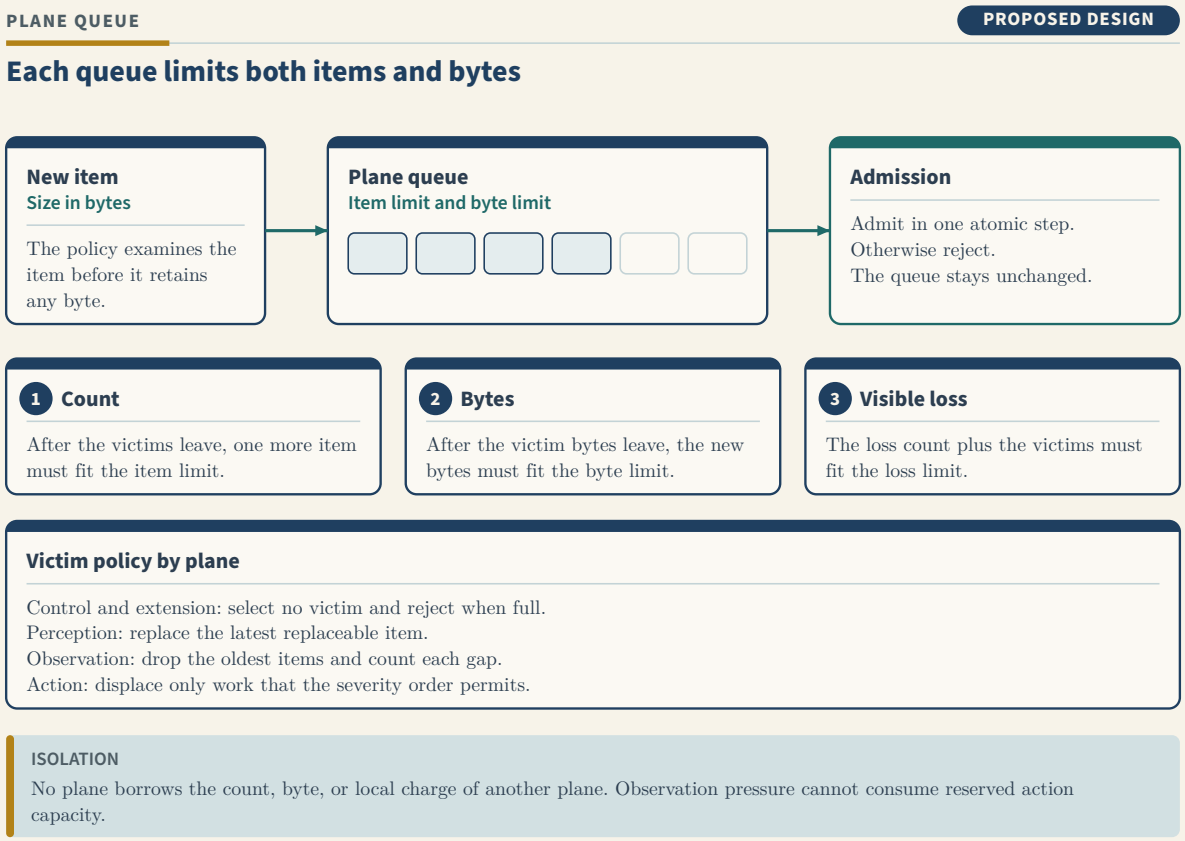


Figure 8: Plane queue admission. Count and byte limits both apply before any byte is retained.

11 Session lifecycle

Status: proposed.

One bounded slot serializes open, replacement, and close for one namespace ν. A namespace is the triple of realm, session kind, and logical ID. Figure 9 shows the states.

11.1 Single flight

Let $\mathcal{O}_\nu$ be the lifecycle records of namespace ν. The number of busy records is

$$N_{\text{busy}}(\nu) = \sum_{u \in \mathcal{O}_\nu} \mathbf{1}[\text{state}(u) \in \{\textsc{pending}, \textsc{ambiguous}\}]\,. \tag{32}$$

The owner has one active slot for ν. Each entry into a busy state first proves that the slot is free. Therefore

$$0 \le N_{\text{busy}}(\nu) \le 1. \tag{33}$$

A mutex cannot give this bound across processes. A shared writer must also hold one fenced writer epoch.

11.2 Transition order

Let c be the immutable backend coordinate of one operation. The reservation transition makes the slot, the result cell, the prepared context, and the prior lineage durable before any external work starts:

$$\text{reserve} \prec \text{external work}, \tag{34}$$

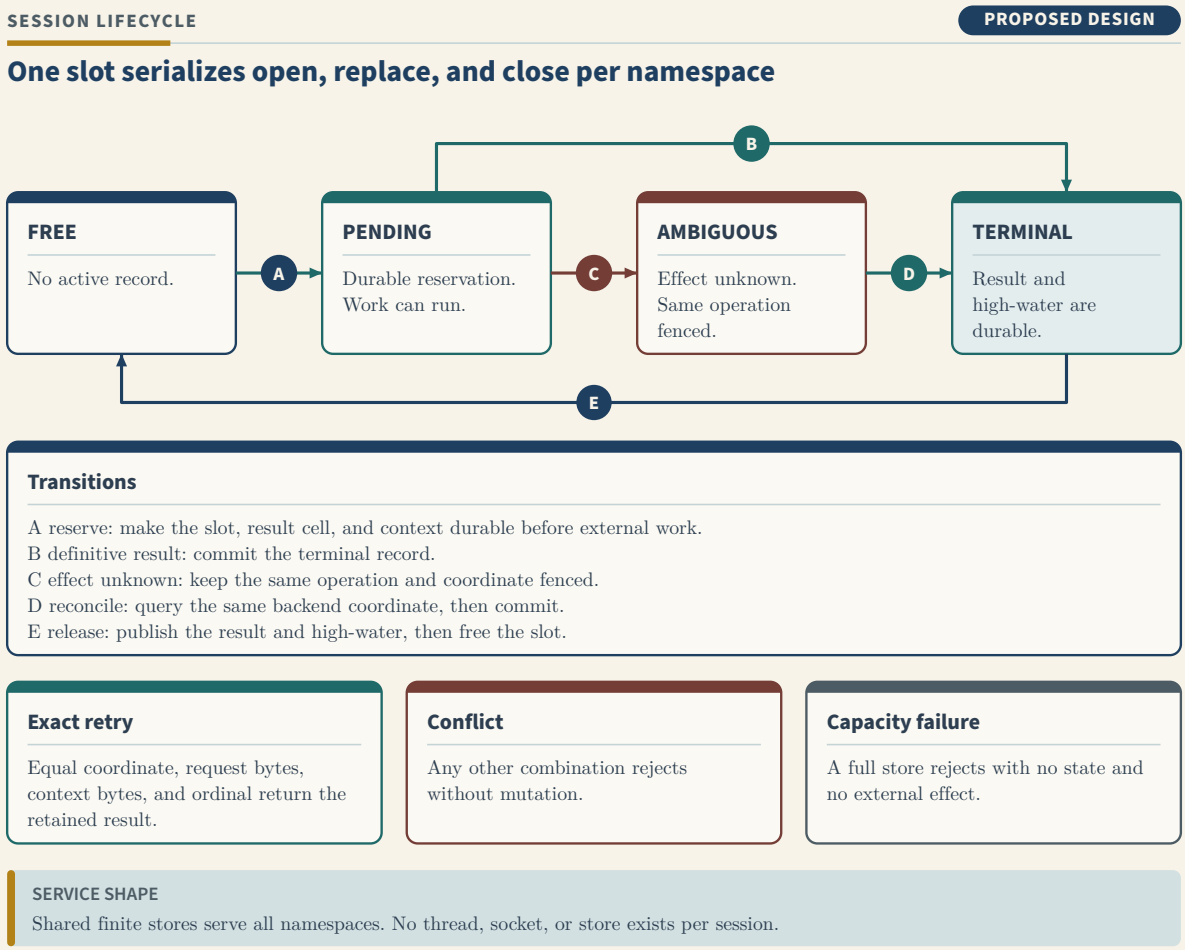


Figure 9: Session lifecycle owner. Terminal commit publishes the result and high-water before it frees the slot.

where $a \prec b$ means that a commits before b starts. A capacity failure therefore leaves no partial slot and no external effect. An unknown effect moves the same record from PENDING to AMBIGUOUS with the same c. Reconciliation queries that c and then commits the record. Terminal commit has the order

$$\text{publish}(\text{result},\ \text{high-water}) \ \prec\ \text{free}(\text{slot}). \tag{35}$$

A later operation cannot see a free slot without also seeing the retained result and the advanced high-water.

11.3 Exact retry

A retained operation is the tuple (c_r, m_r, z_r, o_r) of coordinate, exact request bytes, exact admission-context bytes, and issuer ordinal. A retry (c, m, z, o) is exact only when

$$(c, m, z, o) = (c_r, m_r, z_r, o_r). \tag{36}$$

The comparison of m and z is byte for byte. A digest can find the candidate record. The receiver still compares the retained bytes before it returns or mutates anything. Any unequal component is a conflict, which rejects without mutation.

11.4 Checked ordinals

For one issuer epoch, let $\hat{o} \in \mathbb{N}_{>0}$ with $\hat{o} \le W$. The assignable ordinals and the high-water sentinel are

$$o \in \{1, \dots, \hat{o}\}, \qquad o_{\text{hw}} \in \{0, 1, \dots, \hat{o}\}. \tag{37}$$

The value $o_{\mathrm{hw}} = 0$ means that the epoch has committed no ordinal. A successor exists only when $o_{\mathrm{hw}} < \hat{o}$. Then

$$o_{\mathrm{next}} = \mathrm{cadd}(o_{\mathrm{hw}}, 1) = o_{\mathrm{hw}} + 1, \qquad o_{\mathrm{hw}} < o_{\mathrm{next}} \le \hat{o}. \tag{38}$$

Checked addition cannot fail here because $o_{\mathrm{hw}} + 1 \le \hat{o} \le W$. When $o_{\mathrm{hw}} = \hat{o}$, the epoch seals. An input with $o \le o_{\mathrm{hw}}$ returns the retained result, a retired result, or unavailability. It never executes again.

11.5 No service per session

Shared fixed-capacity tables serve all namespaces. The target is

$$N_{\mathrm{threads/session}} = N_{\mathrm{sockets/session}} = N_{\mathrm{stores/session}} = 0. \tag{39}$$

Session data still exists. It lives in the bounded shared stores of equation (22).

12 Streams, positions, and fixed layouts

Status: proposed.

An accepted stream declaration compiles one immutable context. The context binds the route, publisher incarnation, epoch, frame class, encoding, layout, position range, replay profile, and overload policy.

12.1 Positions

For one stream epoch, let $\hat{p} \in \mathbb{N}_{>0}$ with $\hat{p} \le W$:

$$p \in \{1, \dots, \hat{p}\}, \qquad p_{\mathrm{hw}} \in \{0, 1, \dots, \hat{p}\}. \tag{40}$$

A successor exists only when $p_{\mathrm{hw}} < \hat{p}$, and then

$$p_{\mathrm{next}} = \mathrm{cadd}(p_{\mathrm{hw}}, 1) = p_{\mathrm{hw}} + 1, \qquad p_{\mathrm{hw}} < p_{\mathrm{next}} \le \hat{p}. \tag{41}$$

Otherwise the owner seals the epoch. It never wraps, saturates, or opens a new epoch by itself. The allocator commits each position before encode, queue, or transport work. A later failure leaves a visible gap. With $\mathcal{P}_{\mathrm{asg}}$ the assigned and $\mathcal{P}_{\mathrm{pub}}$ the published positions,

$$\mathcal{P}_{\mathrm{pub}} \subseteq \mathcal{P}_{\mathrm{asg}}, \qquad \mathcal{P}_{\mathrm{gap}} = \mathcal{P}_{\mathrm{asg}} \setminus \mathcal{P}_{\mathrm{pub}}. \tag{42}$$

Assignment is strictly increasing, so no gap position can later name a different payload.

12.2 Fixed-layout frames

A prepared frame has a header of b_{hdr} bytes, n_v numeric values of w_{val} bytes each, and n_g availability groups. Each group has one availability bit, so the bitmap takes

$$a(n_g) = \left\lceil \frac{n_g}{8} \right\rceil \tag{43}$$

bytes. A sensor layout has $n_g \ge 1$, and its nonempty groups partition every sensor value exactly once. A non-sensor layout has $n_g = 0$. The frame length is

$$\ell_{\mathrm{fix}} = b_{\mathrm{hdr}} + a(n_g) + n_v w_{\mathrm{val}}. \tag{44}$$

For $0 \le n_v \le \hat{n}_v$ and $0 \le n_g \le \hat{n}_g$, the ceiling function is nondecreasing, so

$$\ell_{\text{fix}} \le b_{\text{hdr}} + \left\lceil \frac{\hat{n}_g}{8} \right\rceil + \hat{n}_v w_{\text{val}}. \tag{45}$$

The receiver checks n_v and the complete length with checked arithmetic before it decodes any value. Names, units, ranges, and offsets live in the prepared layout, not in each frame.

The bitmap uses bit g for group g, least significant bit first. A set bit means available. High padding bits are zero. An available group holds finite, in-range values. An unavailable group forbids caller values, and the decoder returns typed unavailability, never a placeholder value. The frame digest covers the header, the bitmap, and every value byte. A received zero, an unavailable group, a malformed frame, a transport gap, and a lost frame stay distinct.

For a direct linear decode, the work is the structural operation count

$$\omega(n_v) = \omega_0 + \omega_1 n_v, \tag{46}$$

with fixed counts ω_0 for the header and ω_1 for each value. It is not a time measurement.

13 Simulation steps

Status: proposed.

Opening, configuration, and close stay bounded JSON control operations. Repeated steps use one prepared fixed-layout request and response pair. Figure 10 shows the exchange.

13.1 Step window

Let $\hat{n}_{\text{step}} \in \mathbb{N}_{>0}$ be the window capacity and n_{open} the number of reserved requests that are not terminal. Admission requires

$$n_{\text{open}} < \hat{n}_{\text{step}}, \tag{47}$$

which leaves one free slot, and after reservation $n_{\text{open}} + 1 \le \hat{n}_{\text{step}}$. Let $\hat{b}_{\text{req}}$, $\hat{b}_{\text{res}}$, and $\hat{b}_{\text{meta}}$ bound one request, one response, and one metadata record, and $\hat{b}_{\text{win}}$ the fixed window overhead. With both directions reserved in advance,

$$M_{\text{win}} \le \hat{b}_{\text{win}} + \hat{n}_{\text{step}}(\hat{b}_{\text{req}} + \hat{b}_{\text{res}} + \hat{b}_{\text{meta}}). \tag{48}$$

Each active simulation session owns at most one window, and each window is charged to one distinct session slot. The number of windows is therefore $n_{\text{win}} \le \hat{n}_{\text{ses}}$, and

$$M_{\text{step}} = \sum_{w=1}^{n_{\text{win}}} M_{\text{win}}^{(w)} \le \hat{n}_{\text{ses}} \left[\hat{b}_{\text{win}} + \hat{n}_{\text{step}}(\hat{b}_{\text{req}} + \hat{b}_{\text{res}} + \hat{b}_{\text{meta}}) \right]. \tag{49}$$

The local charge has the same form with one runtime unit: $J_{\text{step}} \le \hat{n}_{\text{ses}}[J_{\text{win}} + \hat{n}_{\text{step}} J_{\text{slot}}]$. A close can move a terminal response into the retained-result store. That move and the window release form one atomic step.

13.2 Execution order and correlation

Let p_{next} be the next executable position of one generation. A request at position p starts only when

$$p = p_{\text{next}}, \tag{50}$$

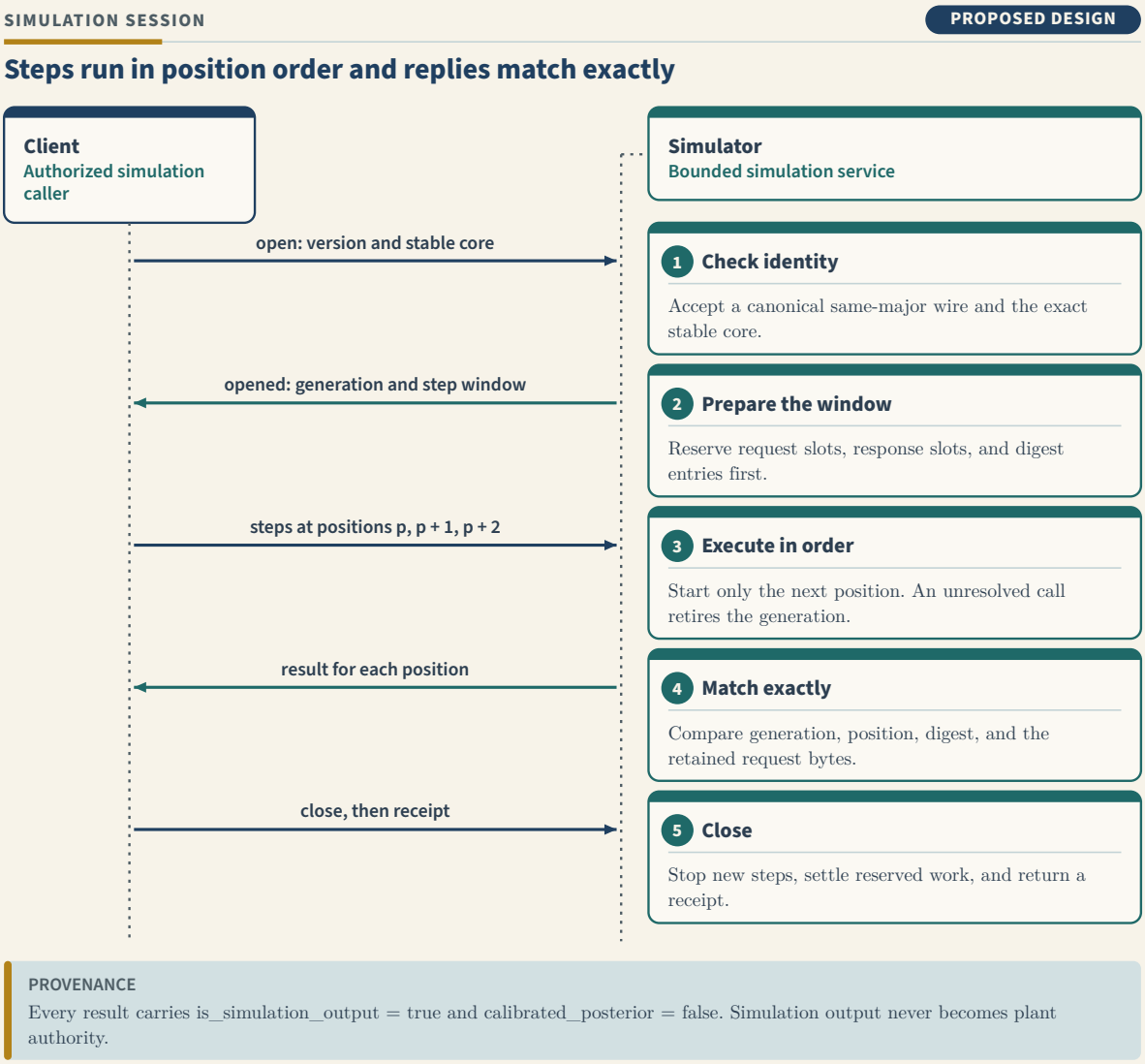


Figure 10: Simulation session. Steps run strictly in position order. Each result matches by generation, position, and request digest.

and definite completion sets $p_{\text{next}} \leftarrow \text{cadd}(p_{\text{next}}, 1)$. If that value is undefined, the owner seals the generation. An unresolved backend call retires the generation. Arrival order never decides execution order.

The retained response is

$$(\text{generation},\ p,\ H(m_p),\ \text{result}), \tag{51}$$

where m_p is the exact request at position p. A response satisfies only a waiter with the same generation, position, digest, and exact request bytes. A reply never takes the next waiter in arrival order. Every result carries `is_simulation_output=true` and `calibrated_posterior=false`.

14 Plant command admission

Status: proposed.

The body issues live authority and command freshness. A serialized lease grants no authority. Arrival never starts or extends a remote command lifetime. Figure 11 shows the order.

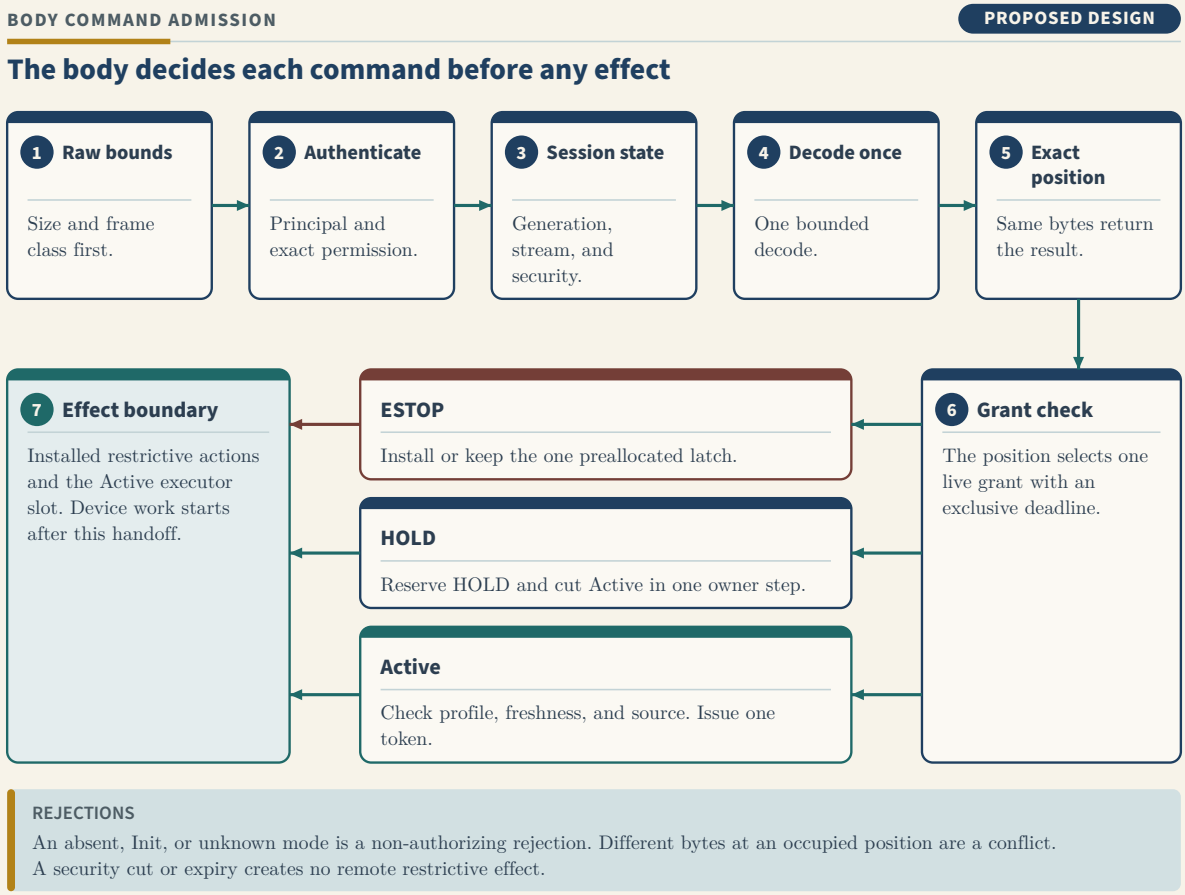


Figure 11: Body command admission. ESTOP, HOLD, and Active follow different paths to the effect boundary.

14.1 Freshness grant

Let t_{iss} be the receiver tick at which the body creates a freshness grant, and $\hat{\Delta} \in \mathbb{N}_{>0}$ the profile lifetime in ticks of the same clock incarnation. The exclusive deadline is

$$d = \mathrm{cadd}(t_{\mathrm{iss}}, \hat{\Delta}). \tag{52}$$

If d is undefined, the body rejects the grant before it becomes visible. At tick t of the same incarnation, the grant is live only when

$$t_{\mathrm{iss}} \le t < d. \tag{53}$$

An arrival tick is audit evidence only. Replacing t_{iss} with it would create a new lifetime after network delay. A grant covers one contiguous position range $[p_0, p_1]$ with $1 \le p_0 \le p_1 \le \hat{p}$, and admission also requires

$$p_0 \le p \le p_1. \tag{54}$$

The position selects the grant. The frame repeats no grant, lease, or lifetime.

14.2 Active predicate

An Active command passes software admission only when all six predicates hold:

$$\phi_{\text{active}} = \phi_{\text{current}} \wedge \phi_{\text{lease}} \wedge \phi_{\text{grant}} \wedge \phi_{\text{stream}} \wedge \phi_{\text{plant}} \wedge \phi_{\text{exec}}. \tag{55}$$

The six predicates have these meanings:

- ϕ_{current}: the security snapshot and the session are current.
- ϕ_{lease}: the exact lease is live.
- ϕ_{grant}: equations (53) and (54) hold.
- ϕ_{stream}: the epoch and the position pass the no-reuse checks.
- ϕ_{plant}: the plant profile accepts the channel, arity, units, and range.
- ϕ_{exec}: the bounded executor slot accepts the exact command.

The predicate stops at software admission.

14.3 Restrictive modes

HOLD needs an explicit known mode and a live holder. It follows ordinary stream order. ESTOP can use the separately enrolled emergency lane when policy permits it. Neither mode carries remote values. Their actions come from the installed plant profile. An absent, `Init`, or unknown mode is a non-authorizing rejection. A rejected remote request cannot report that a remote HOLD or ESTOP took effect.

15 Effect boundary and closed-loop limits

Status: boundary proposed, stability arithmetic exact.

The body owner creates one bounded admission token that binds the exact command, plant profile, generation, authority state, freshness state, and executor incarnation. The actuator interface accepts no raw map, transport message, or serialized lease.

Let e_a be the software admission event and e_x a later physical effect. NCP evidence can record e_a. It cannot establish $e_a \Rightarrow e_x$. Device, mechanical, electrical, and plant-local safety systems stay outside the protocol. Each deployment needs its own safety case and independent interlock.

15.1 Timing on one clock

Let t_s, t_a, and t_d be the sample, admission, and disposition ticks of one body clock incarnation. When the events exist, the latencies are

$$\lambda_a = t_a - t_s, \qquad \lambda_d = t_d - t_s, \tag{56}$$

and an admitted action with a later disposition satisfies $0 \le \lambda_a \le \lambda_d$. A rejected action can have no admission event, so λ_a can be undefined. An absent event never becomes zero. Differences between unrelated publisher and receiver clocks are not equivalent measurements.

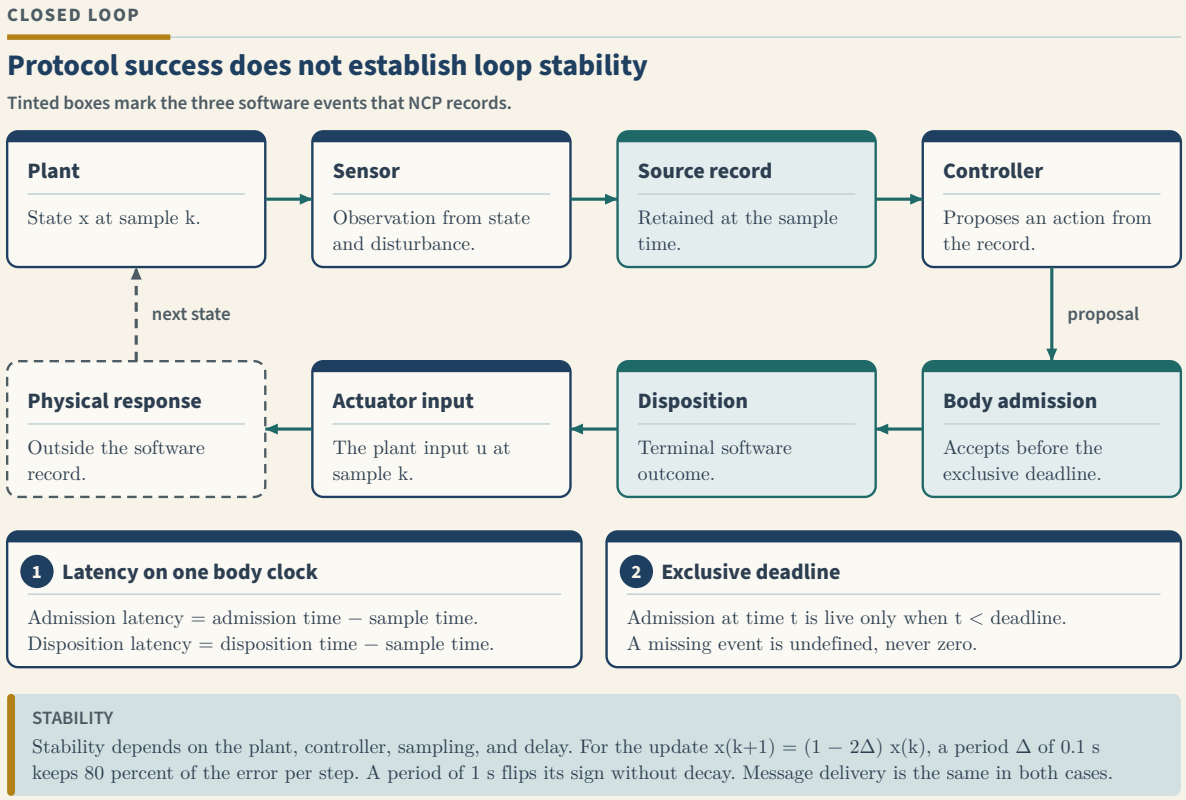


Figure 12: One feedback cycle. NCP records three software events. It cannot measure the physical response.

15.2 Stability needs a plant model

Consider a linear plant $\dot{x}(t) = A_c x(t) + B_c u(t)$ with state $x \in \mathbb{R}^{n_x}$ and input $u \in \mathbb{R}^{n_u}$. Assume exact state samples, no delay, no saturation, and an input held constant over each sample period $\Delta > 0$. Then

$$x_{k+1} = \Phi(\Delta) x_k, \qquad \Phi(\Delta) = e^{A_c \Delta} + \left(\int_0^{\Delta} e^{A_c \theta} \, d\theta \right) B_c K \tag{57}$$

for state feedback $u_k = K x_k$ with gain matrix K. For a fixed period, the sampled loop is asymptotically stable exactly when the spectral radius satisfies $\rho(\Phi(\Delta)) < 1$.

For a scalar integrator $\dot{x} = u$ with $u_k = K x_k$ and $K = -2\,\mathrm{s}^{-1}$, the factor is

$$\Phi(\Delta) = 1 - 2\Delta. \tag{58}$$

At $\Delta = 0.1$ s, $\Phi = 0.8$: each step keeps 80 percent of the error. At $\Delta = 1$ s, $\Phi = -1$: the error changes sign with no decay. Message delivery is the same in both cases, so protocol success cannot show stability.

A variable period needs a stronger argument than one spectral radius for each period. One sufficient condition is a common pair of symmetric positive-definite matrices P and Q with $\Phi(\delta)^{\mathsf{T}} P \Phi(\delta) - P \preceq -Q$ for every admitted period δ. This report supplies no such pair for any ecosystem controller.

16 Extensions and the complete state envelope

Status: proposed.

One extension message holds one complete bounded canonical-JSON envelope. Let ℓ_{env} be its length and n_{att} its number of attachment references. Admission requires

$$0 < \ell_{\mathrm{env}} \le \hat{\ell}_{\mathrm{env}}, \qquad 0 \le n_{\mathrm{att}} \le \hat{n}_{\mathrm{att}}, \qquad \sum_{i=1}^{n_{\mathrm{att}}} \ell_{\mathrm{att},i} \le \hat{\ell}_{\mathrm{att}}, \tag{59}$$

where $\ell_{\mathrm{att},i}$ is the declared length of attachment i. The installed manifest selects one closed schema. It rejects unknown members, duplicate keys, non-canonical numbers, and inline attachment bytes before any callback work.

Attachments come only from enrolled stores. The manifest fixes the exact HTTPS origin, address set, TLS identity, credential source, timeouts, media types, and byte limits. The wire cannot supply a URL, local path, or redirect target. The receiver reserves all resources first. It checks each declared length and digest before semantic use. A cut during a fetch prevents the callback. NCP 1.0 defines no generic chunk-reassembly protocol.

Let $\hat{n}_{\mathrm{ext}}$ bound the envelope slots of one activation. Let $\hat{b}_{\mathrm{ext}}$ bound the larger of the active and terminal metadata of one slot, and let $\hat{b}_{\mathrm{act}}$ be the fixed charge of one activation. Each activation consumes one prepared-context unit, so the activation count is at most $\hat{n}_{\mathrm{ctx}}$:

$$M_{\mathrm{ext}} \le \hat{n}_{\mathrm{ctx}} \left[\hat{b}_{\mathrm{act}} + \hat{n}_{\mathrm{ext}} (\hat{\ell}_{\mathrm{env}} + \hat{b}_{\mathrm{ext}}) \right]. \tag{60}$$

A context stays charged while any slot or terminal record refers to it, so a retained record cannot escape the factor $\hat{n}_{\mathrm{ctx}}$. The local charge J_{ext} has the same form.

16.1 Complete bounded state

Let $\mathcal{Y} = \{\mathrm{rx}, \mathrm{json}, \mathrm{att}, \mathrm{parse}, \mathrm{callback}\}$ name receive scratch, JSON arenas, attachment storage, extension parser arenas, and callback buffers. Each domain y has a portable ceiling $\hat{b}_y$, with fixed charge $\hat{b}_{Y,0}$:

$$M_{\mathrm{aux}} \le \hat{b}_{Y,0} + \sum_{y \in \mathcal{Y}} \hat{b}_y. \tag{61}$$

All domains are disjoint, so the complete NCP-owned state satisfies

$$M_{\mathrm{NCP}} \le M_{\mathrm{prep}} + M_{\mathrm{step}} + M_{\mathrm{ext}} + M_{\mathrm{aux}}, \tag{62}$$

with the same form for the local charge. The sum applies no alias discount. A borrowed view is charged to its owner, and a copy is a second charge. During a transfer between owners, both stay reserved until the destination commits. The bound excludes code, thread stacks, device memory, and unrelated process state.

17 Identity and ecosystem

Status: proposed, role qualifications NOT RUN.

17.1 Identity domains

Figure 13 separates the identities. A native session needs a canonical same-major wire version and the exact stable-core identity. A wire version has one or two canonical decimal components. The value `1` means `1.0`. The compact protobuf hash, the complete contract digest, the package version, and a release tag cannot open a native session by themselves.

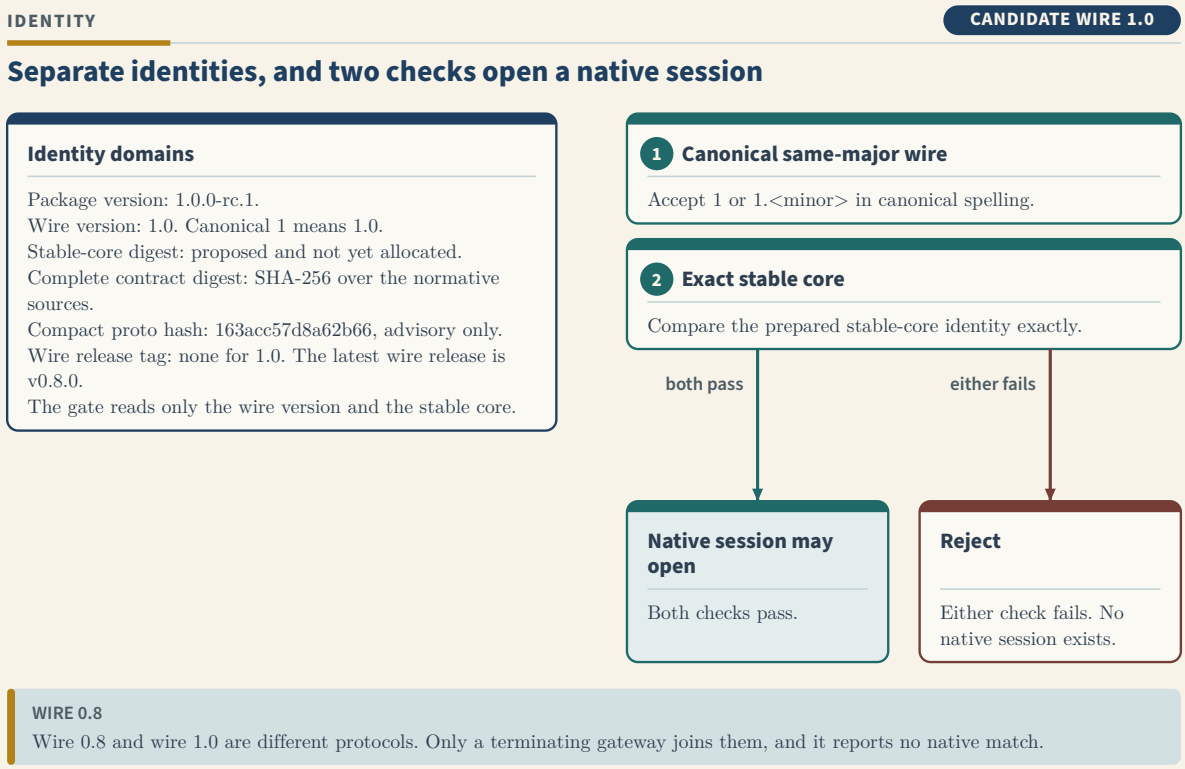


Figure 13: Identity domains and the native session gate.

17.2 Dependency direction

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the repository dependency graph. An edge $(v, w) \in \mathcal{E}$ means that v depends on w. Let v_{NCP} be NCP and $\mathcal{C}$ the consumer repositories that integrate NCP. Write $v \rightsquigarrow w$ for a directed path from v to w. The required direction is

$$\forall c \in \mathcal{C} : \ (c, v_{\mathrm{NCP}}) \in \mathcal{E} \ \wedge \ \neg(v_{\mathrm{NCP}} \rightsquigarrow c). \tag{63}$$

The first condition makes each consumer use the NCP package instead of a copy of the protocol files. The path condition also forbids an indirect dependency of NCP on a consumer. A consumer-specific fork therefore cannot become an NCP requirement.

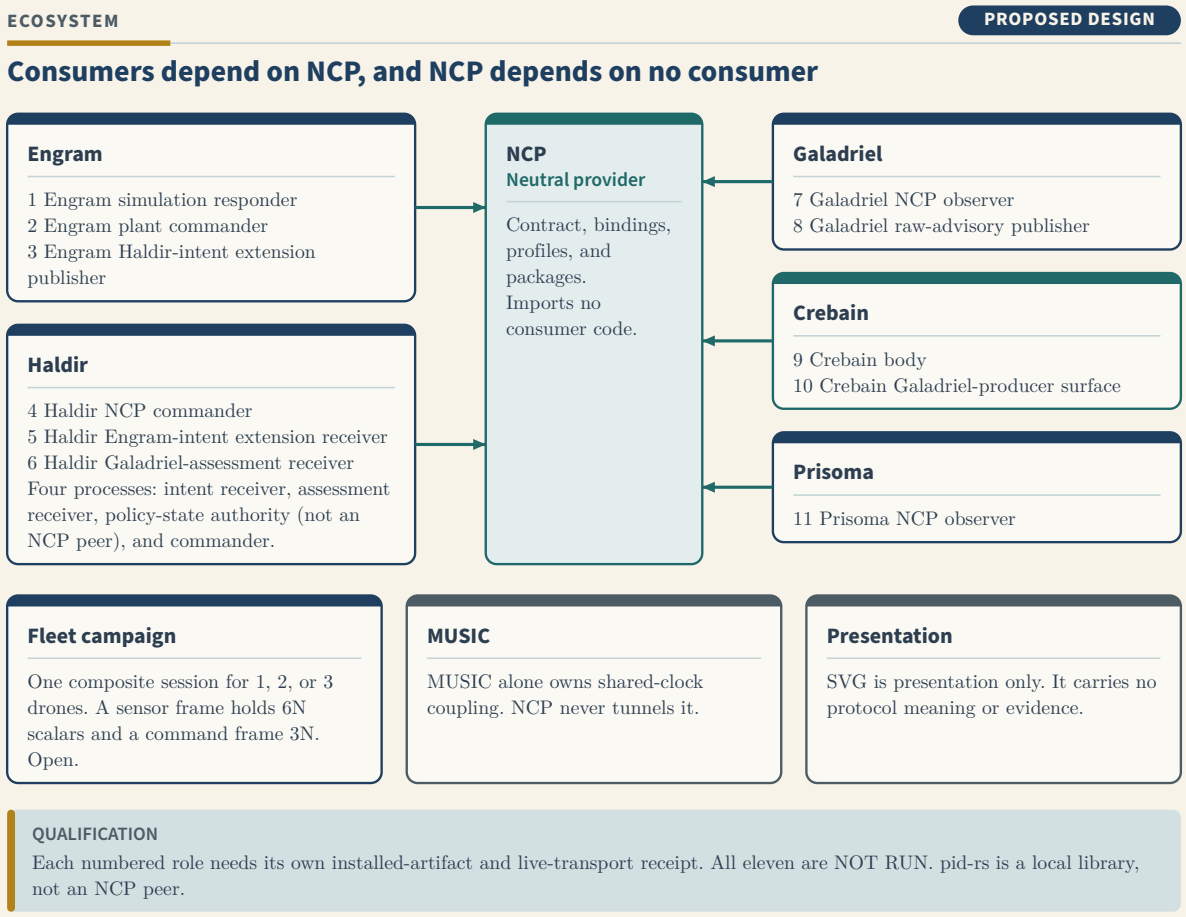


Figure 14: Ecosystem dependency direction and the eleven role subjects.

17.3 Roles and processes

Figure 14 numbers the eleven role subjects. Each needs its own installed-artifact and live-transport receipt. All eleven are **NOT RUN**. The library `pid-rs` is not an NCP peer and receives no role receipt.

Integrated Haldir uses four separate processes. The intent receiver owns intent transport, replay, and attachment fetch. The assessment receiver owns assessment ingress and raw evidence. The policy-state authority owns profiles, base policy, and freshness grants, and it is not an NCP peer. The commander owns conversion into NCP commands, streams, and publication. No process holds a credential of another role. Standalone Gate is a separate deployment mode, not a fifth process.

17.4 Fleet profile

The selected fleet profile uses one composite session for each run with $N \in \{1, 2, 3\}$ drones. One sensor frame holds $6N$ values: position and velocity in east, north, and up order for each drone. One command frame holds $3N$ acceleration values. Each drone has one availability group, so the bitmap has N bits. A qualification run must exercise every bitmap value from 0 to $2^N - 1$, which is every subset of unavailable drones. Group g maps to command values $3g$, $3g + 1$, and $3g + 2$. An unavailable drone needs its installed restrictive values. A mismatch rejects the whole command.

One NEST 3.9 kernel advances the complete fleet at a resolution of 0.1 ms. Each controller epoch lasts 20 ms. The qualification seed is `20260826`. Historical fleet evidence from the older Engram host interface cannot satisfy a native role, transport, or release qualification. The fleet campaign is open.

MUSIC, a library for coupling simulators on a shared clock, alone owns shared-clock coupling. NCP does not implement, tunnel, or replace MUSIC time grants, barriers, or ticks. The fleet profile uses independent clocks. SVG files are presentation only. They carry no protocol meaning or runtime evidence.

18 Current candidate runtime and gaps

Status: implemented runtime with known gaps.

Figure 15 shows the current compatibility governor. It returns bounded wire-shape candidates. It does not authenticate a sender, issue a lease, load a plant profile, or prove a physical effect.

For a configured positive timeout T, sensor data becomes stale when its age reaches $\min(T, 60\ \text{s})$. The governor latches ESTOP when the time since the last finite sensor timestamp reaches $\min(20 \min(T, 60\ \text{s}),\ 60\ \text{s})$. For example, $T = 0.5$ s gives 10 s, and $T \geq 3$ s gives 60 s.

Table 7 lists the main gaps between this runtime and the proposed target. Each row names the source that shows the gap. The task ledger lists the complete set.

Table 7: Verified gaps at commit `e83df54`.

Area	Current behavior	Source
Secure ingress	Pinned Zenoh 1.9.0 does not expose the verified peer principal to the callback. `production-secure` fails closed.	`ncp-zenoh/src/lib.rs`
Actor capability	`AuthenticatedActor` has public fields, so any caller can construct it.	`ncp-core/src/security.rs`
Grant scope	A principal grant covers a complete plane, with no exact route, audience, frame class, or operation list.	`ncp-core/src/security.rs`
Body owner	No single owner orders security cuts, leases, streams, restrictive effects, dispositions, and executor admission.	`ncp-core/src/transport.rs`
Lease time	`NeuroControlLoop` stores a host-supplied lease with no receiver deadline.	`ncp-core/src/transport.rs`
Tick result	`tick` returns the same `CommandFrame` type for a local pre-sensor output and a transport-admitted command.	`ncp-core/src/transport.rs`
Send outcome	`CommandSendOutcome` binds a stream position, not the payload digest or owner incarnation.	`ncp-core/src/transport.rs`
Mode default	An omitted mode decodes as HOLD. The target rejects it.	`ncp-core/src/messages.rs`
Session lineage	The TypeScript client retires the local generation before an open result is definite.	`ncp-ts/src/client.ts`
Reply order	The WebSocket binding matches replies in request order.	`ncp-ts/src/ws.ts`
RPC isolation	A synchronous Zenoh handler cannot be preempted, so its permit can stay occupied.	`ncp-zenoh/src/lib.rs`
Compact data	No compact data-plane encoding, availability bitmap, durable generation issuer, or disposition journal exists.	`KNOWN_LIMITATIONS.md`

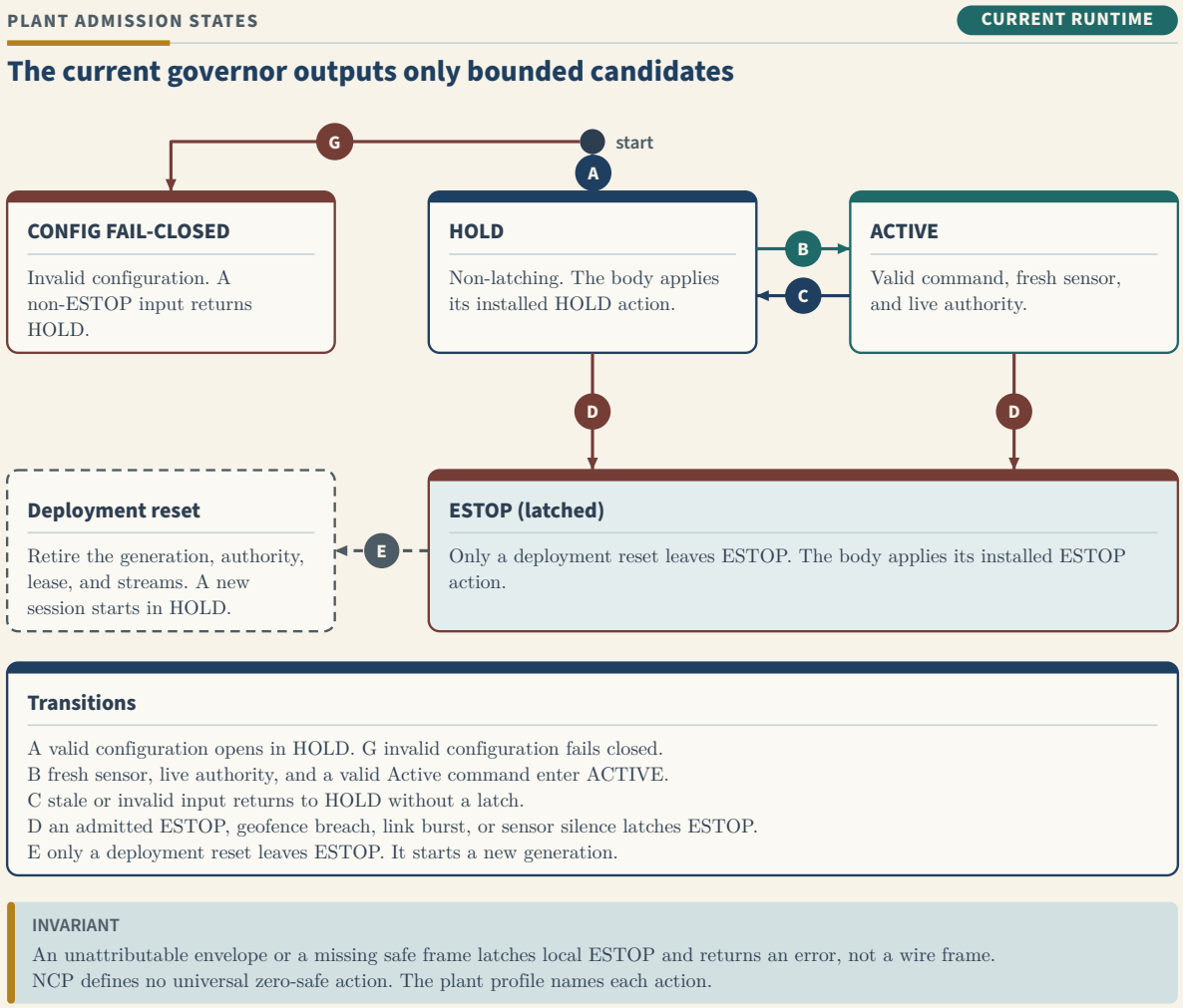


Figure 15: Current plant-admission states of the compatibility governor.

19 Ten-lens review

Status: maintainer review, not independent review.

The project reviews each change through ten lenses from `docs/handoff/NCP_V1_0_ECOSYSTEM_FINALIZATION_BLUEPRINT.md`. Table 8 applies them to the design in this report. This review does not satisfy the independent review floor.

Table 8: Ten-lens review of the design.

Lens	Design answer	Open evidence
1 Contract and semantics	One normative source list with a complete SHA-256 digest. Closed types in the modular SDK.	The rebaseline and identifier allocation for the target.
2 Security and authority	Verified principal, default-deny manifest, body-issued lease. Unknown values grant nothing.	Receiver-minted capability. Live mTLS, rotation, and revocation are **NOT RUN**.
3 Safety and plant boundary	The body is the final software authority. The plant profile names each restrictive action.	A plant safety case and interlock for each deployment.
4 Distributed systems	Exact retry, single flight, and retirement after ambiguity. Positions never repeat.	Cross-process fencing and durable atomicity. Fault and soak campaigns are **NOT RUN**.
5 Resource and real-time bounds	Every count has a byte bound. Checked arithmetic fails closed.	Selected values and measured profiles. The timing campaign missed every deadline.
6 Interoperability and migration	Two independent modular implementations. Wire 0.8 joins wire 1.0 only through a terminating gateway.	Two installed non-Rust live peers are **NOT RUN**.
7 Science and statistics	Simulation output stays uncalibrated. The forecast study reports its null result.	Model or policy benefit needs new frozen studies.
8 Implementation and operations	One entry point for each production path. Gaps, drops, and ambiguity stay visible.	The gaps in Table 7.
9 Verification and evidence	Mandatory corpus, cross-language controls, and Lean-checked arithmetic.	External and independent gates are **NOT RUN**.
10 Lifecycle and governance	Consumers depend on NCP. No release authority comes from local evidence.	Registry ownership, signatures, provenance, and release authorization.

20 Evidence classes and release gates

Status: release-blocked.

No release gate has evidence above class 3 of Figure 2.

The machine policy `contract/release-gates.v1.json` names twelve pre-release gates. Every gate must refer to one immutable source, one normative digest, and exact package and environment identities. Table 9 gives their state. No gate has release-bound evidence.

Two checks run only after publication: an install smoke test and an emergency revocation exercise. They do not block the first tag.

The proto comment conflict concerns the predictive-horizon bound. Let ttl be the command lifetime and dt the horizon step, both in positive milliseconds. The comment in `proto/ncp.proto` reads $N \le \mathrm{ttl}/\mathrm{dt}$. The receiver expires a command when the elapsed time reaches $\min(\mathrm{ttl}, 60{,}000)$, so a step at that boundary never executes. When the ratio is finite, the intended bound is

$$N \le \min(\lceil \min(\mathrm{ttl}, 60{,}000)/\mathrm{dt} \rceil - 1,\ 65{,}536).$$

A correction changes the complete contract digest, so it must follow the rebaseline workflow.

Table 9: Pre-release gates of the 1.0 candidate.

Gate	Class	State
`normative-contract`	Local	Open: a proto comment conflict blocks it.
`zero-skip-conformance`	Local	Local pass at a historical source only.
`live-mtls-acl-rotation-revocation`	External	**NOT RUN**
`two-independent-non-rust-live-peers`	External	**NOT RUN**
`fault-backpressure-restart-soak`	External	**NOT RUN**
`fuzz-sanitizer-duration`	External	**NOT RUN**
`performance-resource-profile`	External	**NOT RUN**
`installed-package-matrix`	External	**NOT RUN**
`registry-namespace-ownership`	External	**NOT RUN**
`consumer-certification`	External	**NOT RUN**
`independent-clean-room-reproduction`	External	**NOT RUN**
`signed-sbom-provenance`	External	**NOT RUN**

A How this report is checked

Status: local publication checks.

The script `scripts/check_ncp_system_design_pdf.sh` rebuilds this report. It does these steps:

1. Convert each light SVG figure to PDF with a private font configuration.
2. Build the report with LuaLaTeX and fixed time inputs.
3. Reject LaTeX warnings, overfull boxes, underfull boxes, and undefined references.
4. Require embedded fonts, A4 pages, and no JavaScript.
5. Check that the equation audit covers every numbered equation in order.
6. Embed a domain-separated SHA-256 commitment over the report sources.
7. Compare the rebuilt PDF with the committed PDF.

The generator `scripts/gen_diagrams.py` owns every figure. It writes a light and a dark SVG of each figure to `docs/diagrams/`. This report uses the light SVGs. The Markdown documents show the SVG that matches the reader's color scheme. It measures each label with the advance widths of the exact font files. It rejects each of these defects:

- a label that does not fit its box, or two labels that overlap,
- a connector that does not end on its boxes, or that crosses another box,
- text contrast below 4.5:1, and
- an SVG that breaks the accessibility contract.

The figures stay vector graphics in this report, so they keep sharp edges at every zoom level.

The Lean 4 file `docs/publication/lean/NcpDesign.lean` checks the discrete arithmetic of this report. It covers equations (1), (2), (4)–(11), (13)–(15), (18), (19), (22), (27), (28), (30), (38), (41), (43)–(45), (53), and (58). It also checks the fleet command groups, the silence deadline, the horizon count, and the sample and byte totals in Table 5. The script `scripts/check_ncp_design_lean.sh` runs the file with Lean 4.23.0. The file uses Lean core only and rejects unchecked constructs.

These checks show artifact integrity and arithmetic. They do not validate the protocol or the scientific and safety claims that NCP excludes.

B Equation audit

Table 10 accounts for every numbered equation. The right column states the meaning, the assumptions, and the behavior when an input is out of range.

Table 10: Equation audit.

Equations	Unit	Meaning, assumptions, and failure behavior
(1)	integer	Checked addition is partial. An undefined sum rejects or seals and never wraps.
(2)–(3)	sequence	Sequence numbers stay exact in binary64. Each request names the last admitted outcome. A mismatch rejects.
(4)–(6)	bytes	Chunks tile the payload with no gap or overlap. At most 256 chunks carry one payload. A bad chunk changes nothing.
(7)–(8)	bytes	Logical working extents of one owner and one client. They are reservations, not process memory.
(9)–(10)	samples	Each group of three body ticks holds exactly 400 pressure samples.
(11)–(12)	μs and Hz	The spike window is nonempty when $t_e \geq \Delta_n$ and $\delta < \Delta_n$. A pending window gives no rate.
(13)–(15)	Boolean	One fixed gate order. The first false gate decides, and no later field is read.
(16)–(19)	ticks	Contiguous stage boundaries telescope. Selected ceilings bound the hot path. Cross-incarnation ticks are invalid.
(20)	operations	Allocation and copy counts add over stages. Zero is a target, not a measurement.
(21)–(24)	bytes and local units	Capacity times unit charge plus fixed overhead. Checked partial sums reject a profile before allocation.
(25)–(31)	items and bytes	Victim selection and three checks keep count, bytes, and loss within bounds. A failed check leaves the queue unchanged.
(32)–(33)	records	At most one busy record for each namespace.
(34)–(35)	order	Reservation precedes external work. Publication precedes the release of the slot.
(36)	bytes	Exact retry is tuple equality with byte comparison. Anything else conflicts.
(37)–(38)	ordinal	Zero is the sentinel. Successors increase strictly. Exhaustion seals the epoch.
(39)	count	No thread, socket, or store exists for one session.
(40)–(42)	position	Positions increase strictly and never repeat. A failed publication leaves a visible gap.
(43)–(45)	bytes	The layout bounds the bitmap and the frame length. The length check precedes decoding.
(46)	operations	A structural work count, linear in the value count.
(47)–(49)	requests and bytes	One free slot is required. The session capacity bounds the number of windows.
(50)–(51)	position and bytes	Only the next position executes. A response needs exact request bytes.
(52)–(54)	ticks and position	One body clock, an exclusive deadline, and a contiguous range from one. Overflow rejects the grant.
(55)	Boolean	Six predicates. The result is software admission, not physical effect.
(56)	ticks	Latencies on one clock. An absent event is undefined.
(57)–(58)	dimensionless	Sampled linear feedback. Stability needs $\rho(\Phi) < 1$, which message delivery does not show.
(59)–(60)	bytes and items	Envelope, attachment, and activation bounds. Every activation keeps its context charged.

Equations	Unit	Meaning, assumptions, and failure behavior
(61)–(62)	bytes	Disjoint domains add with no alias discount.
(63)	graph path	Consumers depend on NCP. NCP reaches no consumer, directly or indirectly.

C Glossary

Table 11: Terms and project names.

Term	Meaning
Body	The plant or its simulation. It is the final software authority before an actuator.
Commander	A controller that sends Active commands under a body-issued lease.
Observer	A read-only consumer of an observation projection.
Generation	A server-issued identity of one live session. It is never reused.
Retained outcome	The exact result bytes that an owner keeps until an exact acknowledgement.
ESTOP, HOLD, Active	Plant command modes. ESTOP latches. HOLD does not latch. Active applies values.
Engram	The neural application in the private Paper2Brain repository. It hosts NEST networks.
CREBAIN	A 3D environment for drone simulation and sensor-fusion research. Role identifiers keep the older spelling, such as `Crebain body`.
Prisoma	Experiments that bind forecasts to one pre-action state and compare later outcomes with baselines. It also captures NCP exchanges.
Galadriel	An advisory sensor-consistency monitor for tampering research. An anomaly does not prove an attack.
Haldir	An experimental reference monitor for mission-level plant commands.
NEST	A simulator for spiking neural networks.
Zenoh	A publish-subscribe and query transport.
MUSIC	A library that couples simulators on a shared clock.

Table 12: Task and campaign codes in repository records.

Code	Meaning and source
B01	Independent review of the eleven proposed decision records. Task ledger.
B02	Owner authorization of the deliberate rebaseline. Task ledger.
B03	Allocation of registries, identifiers, error codes, and limits. Task ledger.
X02	The composed ecosystem and fleet campaign. Task ledger.
L1–L10	The ten review lenses of Table 8. Finalization blueprint.
M1	The sensor transfer and timing campaigns. Prisoma records.
E1	The restored-label forecast study. Prisoma records.